



Ministère de l'Éducation Nationale
de l'Enseignement préscolaire & de sport

CENTRE BTS
LYCÉE QUALIFIANT TECHNIQUE - FÈS



RAPPORT DE PROJET DE FIN D'ÉTUDE

Pour l'obtention du diplôme de
Brevet de **T**echnicien **S**upérieur en
Systèmes **É**lectroniques

Sous le thème :

**INTERFACE BUS CAN POUR
COMMANDE DES RETROVISEURS D'UNE
VOITURE**

Réalisé par :

- ✓ REDA HADDAR
- ✓ KAWTHAR DEROUICH
- ✓ LAAOUAR MOHAMMED

Professeur Encadrant :

M. AZOUGAGH

ANNEE ACADEMIQUE : 2023-2024

**B
T
S

S
E**

DÉDICACE :

À nos chers Parents,

Il est difficile de trouver les mots justes pour exprimer toute l'estime, le dévouement et le respect que nous avons pour vous. Malgré cela, nous vous offrons humblement ce travail en signe de gratitude, reconnaissants pour les sacrifices et les efforts considérables que vous avez consentis en notre faveur.

À nos Familles bien-aimées,

Nous tenons à vous témoigner toute notre reconnaissance et notre affection. Votre présence et votre soutien indéfectible ont constitué des piliers solides dans nos vies, et nous vous sommes profondément reconnaissants pour tout ce que vous avez fait.

À nos respectés Professeurs,

Nous désirons exprimer notre profonde gratitude et notre sincère appréciation envers vous. Votre dévouement à l'éducation et votre engagement envers notre réussite académique ont laissé une empreinte indélébile sur nos vies.

À nos amis proches du BTS,

Nous dédions ce rapport à vous tous. Votre présence joyeuse, vos encouragements constants et vos moments de partage ont enrichi cette aventure de manière inestimable. Chacun de vous a contribué à la richesse de notre expérience, et nous vous remercions du fond du cœur Ensemble, nous avons traversé des moments de joie, de défi et d'apprentissage. Cette dédicace témoigne de l'importance que vous avez dans nos vies.

REMERCIEMENT :

Nous voulons exprimer un grand merci à toute l'équipe professionnelle et administrative du centre BTS au lycée technique de Fès, ainsi qu'à nos professeurs de la filière systèmes électroniques qui ont joué un rôle clé dans notre formation. Leurs conseils avisés, leur façon exemplaire d'enseigner et leur disponibilité constante ont été d'une aide inestimable tout au long de notre cursus.

Un remerciement spécial va à notre encadrant de projet, M. AZOUGAGH HOUSSINE, pour son encadrement précieux, ses conseils judicieux et sa disponibilité permanente. Son accompagnement a été crucial dans la réalisation de notre projet, et nous lui sommes infiniment reconnaissants.

Nous tenons à exprimer notre gratitude sincère envers nos proches et toutes les personnes qui nous ont soutenus de près ou de loin. Leurs encouragements, leur sollicitude et leur aide précieuse ont été des moteurs pour mener à bien ce travail. Nous leur sommes profondément reconnaissants pour leur soutien constant tout au long de notre parcours.

Enfin, nous voulons remercier chaleureusement les membres du jury qui ont accepté d'évaluer notre travail. Leurs remarques constructives et critiques ont été d'une grande valeur pour nous, nous permettant de nous améliorer et d'enrichir notre projet.

À tous ceux qui ont contribué de près ou de loin à la réalisation de ce projet, nous vous exprimons notre profonde gratitude. Votre soutien, vos conseils et votre présence ont été des éléments essentiels dans notre parcours, et nous sommes honorés d'avoir pu bénéficier de votre collaboration précieuse. Merci du fond du cœur.

RÉSUMÉ :

Notre projet consiste à créer une interface utilisant le Bus CAN pour contrôler les rétroviseurs d'une voiture. Cette solution permettra une communication efficace entre le système central de commande et les rétroviseurs, améliorant ainsi la réactivité et la précision des ajustements. L'objectif est d'optimiser la gestion électronique des rétroviseurs pour offrir une expérience de conduite plus fluide et sécurisée.

ABSTRACT :

This project entails the development of a Controller Area Network (CAN) interface for the control of car mirrors. The objective is to establish efficient communication between the central control system and the mirrors, enhancing responsiveness and precision in adjustments. The aim is to optimize the electronic management of car mirrors, providing a smoother and more secure driving experience.

CAHIER DE CHARGE :

CHAPITRE I. PRÉSENTATION GÉNÉRALE DU PROJET	9
I. Introduction :	10
1. Contexte du projet :	10
2. Les objectifs de ce projet :	10
II. Présentation du BUS CAN :	11
2. Introduction :	11
2. Origine :	11
3. Principes Généraux de Communication :	11
4.Principe de Fonctionnement :	12
a- Routage des Informations :	12
b- arbitrage :	13
5. Trame de données :	14
CHAPITRE II. ENVIRONNEMENT MATÉRIEL	15
I. Le microcontrôleur :	16
1. Le choix d'un microcontrôleur :	16
2. Arduino Uno :	16
3. Noms et fonctions des broches d'un Arduino Uno :	18
II. AFFICHEUR LCD 20*4 :	19
1. Connection d'un afficheur avec Arduino Uno :	20
2. Noms et fonctions des broches des écrans LCD de caractères :	20
III. Transmetteur \ Récepteur BUSCAN MODULE (MCP2515) :	21
1. Noms des broches du BUSCAN (MCP2515) :	22
2. description des broches :	23
3. Diagramme :	24
IV. interactive 3 positions rotary switch :	25
V. Bouton-poussoir :	26
VI. Module L298 pour commande de moteur DC :	27
1. Les broches de module moteur :	28
2. Connection d'un hacheur avec Arduino :	28
VII. Moteur DC :	29
VIII. Potentiomètre :	30
CHAPITRE III. APPLICATIONS ET RÉALISATION	32
I. Description du système :	33
1. Description du microcontrôleur ARD1 :	33
a) Ça capacité de communication avec le BUS CAN :	33
b) Gestion des boutons de commande des rétroviseurs :	33

c) Traitement des commandes et contrôle des rétroviseurs :	34
d) Intégration dans le système global :	34
2. Description du microcontrôleur ARD2 :	35
c) Capacité de communication avec le BUS CAN :	35
b) Traitement des commandes et contrôle des moteurs des rétroviseurs :	35
c) Intégration dans le système global :	35
3. Schéma de connexion :	36
II. Programme du système :	37
1. Programme du microcontrôleur (Write) ARD1 :	37
2. Programme du microcontrôleur (Read) ARD2 :	40
CHAPITRE IV . LA RÉALISATION ET CIRCUIT IMPRIMÉ	44
I. La Réalisation du ARD1 :	45
1. Introduction :	45
2. Schématique de circuit imprimé :	45
3. Board de circuit imprimé :	46
4. Circuit imprimé :	46
II. La Réalisation du ARD2 :	47
1. Introduction :	47
2. Choix de miroirs :	47
CHAPITRE V. LES LOGICIELS UTILISÉS	49
I. Proteus8 Professional ‘ISIS’ :	50
1. Barre de menus	50
2. Barres d’outils de commande	51
3. Barre d’outils de sélection de mode	51
4. Barre d’outils d’orientation	51
5. Sélecteur d’objets	51
6. Zone de travail ou d’édition des schémas	52
II. Arduino IDE :	52
1. Logiciel :	53
III. Fritzing :	54
IV. Eagle :	55
1. L’interface de contrôle d’eagle :	55
2. La fenêtre d’édition de schéma :	55
3. Barre d’outils :	56
4. Saisie du schéma :	56
CONCLUSION :	58
annexe 1	59
annexe 2	63

LISTE DE FIGURES :

Figure 1: Principe généraux de communication	12
Figure 2: Arbitrage	14
Figure 3: Trame de donnée	14
Figure 4: Arduino Uno	16
Figure 5: Datasheet d'un Arduino Uno	18
Figure 6: afficheur LCD 20*4	19
Figure 7: Connection d'un afficheur avec Arduino Uno	20
Figure 8: datasheet d'un afficheur LCD	20
Figure 9: MCP2515 CAN Module	21
Figure 10: Datasheet d'un BUSCAN (MCP2515)	22
Figure 11: Diagramme	24
Figure 12: interactive 3 positions rotary switch	25
Figure 13: Bouton poussoir	26
Figure 14: Module L298 pour commande de moteur DC	27
Figure 15: datasheet d'un module L298	28
Figure 16: connexion d'un hacheur avec Arduino	28
Figure 17: Moteur DC	29
Figure 18: Potentiomètre	30
Figure 19: Partie 'write'	34
Figure 20: Partie 'read'	35
Figure 21: schéma générale	36
Figure 22: Schématique ARD1	45
Figure 23: Board ARD1	46
Figure 24: Circuit imprimé de ARD1	46
Figure 25: Rétroviser	47
Figure 26: Emplacement des moteurs	48
Figure 27: Câblage avec Les Moteur DC	48
Figure 28: ISIS	50
Figure 29: Arduino IDE	52
Figure 30: Fritzing	54
Figure 31: interface du contrôle de eagle	55
Figure 32: La fenêtre d'édition de schéma	56
Figure 33: La boîte de dialogue	57
Figure 34: Schéma générale en Proteus	59
Figure 35: Les composants utilisés	59
Figure 36: Schéma ARD1	60
Figure 37: Schéma ARD2	61
Figure 38: Schéma générale en Pratique	62

LISTE DE TABLE :

Tableau 1: description des broches.....	23
---	----

ACRONYMES :

CAN	:	Controller Area Network
EMI	:	électromagnétiques
ISO	:	Organisation internationale de normalisation
NRZ	:	No Return to Zero
CSMA-CD	:	Carrier Sense Multiple Access with Collision Detection
AMP	:	Arbitration Message Priority
SOF	:	Start of Frame
CRC	:	Cyclic Redundancy Code
SRAM	:	Static Access Memory
EEPROM	:	Electrically Erasable Programmable Read-Only Memory
LCD	:	Liquid Crystal Display
ARES	:	Advanced Routing and Editing Software
DC	:	Courant continu
ADR	:	Arduino
PWM	:	Pulse Width Modulation

INTRODUCTION GÉNÉRALE :

À l'issue de nos deux années d'études, nous entamons un projet de fin d'études qui nous offre l'occasion d'appliquer les connaissances acquises durant notre formation. Dans ce cadre, nous avons choisi de nous focaliser sur le sujet de l'interface Bus CAN (Controller Area Network) dédiée à la commande des rétroviseurs d'une voiture.

Le Bus CAN est un protocole de communication largement utilisé dans l'industrie automobile, facilitant la communication fiable et efficace entre les différents composants électroniques d'un véhicule. Sa robustesse face aux interférences électromagnétiques et sa capacité à gérer plusieurs points de communication en font une solution idéale pour le contrôle des systèmes embarqués, incluant la commande des rétroviseurs.

Dans le cadre de ce projet, notre objectif est de concevoir et mettre en place une interface Bus CAN spécifiquement dédiée à la commande des rétroviseurs. Nous nous attacherons à créer un circuit électronique et à élaborer un logiciel adapté, permettant un contrôle précis et fiable des rétroviseurs d'une voiture via le Bus CAN. Notre démarche impliquera l'étude de l'architecture du système, la conception du circuit électronique, le développement du code source, et la réalisation de tests approfondis pour évaluer les performances et la fiabilité du système.

Ce projet revêt une importance significative, contribuant à améliorer le confort et la facilité d'utilisation des véhicules grâce à une solution de commande avancée pour les rétroviseurs. De plus, il représente une opportunité pour approfondir nos connaissances sur le Bus CAN et renforcer notre expertise en conception et développement de systèmes embarqués.

CHAPITRE I.

PRÉSENTATION GÉNÉRALE DU PROJET

I. Introduction :

1. Contexte du projet :

Dans le secteur automobile, la gestion des rétroviseurs représente une fonction cruciale, apportant confort et commodité aux conducteurs et passagers. Historiquement, le contrôle des rétroviseurs s'effectuait à l'aide d'interrupteurs situés sur les portières du véhicule. Toutefois, l'avancement technologique a ouvert la voie à une centralisation de cette commande en exploitant le BUS CAN.

2. Les objectifs de ce projet :

Les objectifs du présent projet de PFE résident dans la conception et le développement d'un système de commande des rétroviseurs d'une voiture, exploitant le BUS CAN. L'objectif principal consiste à centraliser et simplifier la gestion des rétroviseurs, en permettant un contrôle précis et efficace à partir d'une unité centrale. Ce projet offre également l'opportunité d'explorer les avantages qu'offre le BUS CAN en termes de communication et d'intégration des divers systèmes électroniques d'un véhicule.

3. Pourquoi nous avons choisi ce projet ?

Ce projet vise à développer une solution innovante pour améliorer la gestion électronique des rétroviseurs automobiles. En utilisant le Bus CAN, notre objectif principal est de concevoir une interface permettant un contrôle précis et efficace des rétroviseurs depuis une unité centrale. Cette approche centralisée offre une réponse rapide et une intégration fluide avec d'autres systèmes électroniques du véhicule, contribuant ainsi à une expérience de conduite plus sûre et plus pratique. Ce projet représente une opportunité passionnante d'explorer les avantages du Bus CAN dans le contexte spécifique de la commande des rétroviseurs, avec le potentiel d'innovation et d'amélioration continue.

4. Justification de l'utilisation du BUS CAN :

L'utilisation du BUS CAN dans ce projet de commande des rétroviseurs se justifie par ses avantages clés. Le BUS CAN offre une communication fiable entre les composants électroniques du véhicule, réduit la complexité du câblage, optimise l'efficacité du système grâce à une gestion précise des données, et favorise une intégration harmonieuse avec d'autres systèmes électroniques. Ces caractéristiques font du BUS CAN le choix optimal pour garantir une commande précise et efficace des rétroviseurs, contribuant ainsi à une expérience de conduite sécurisée et pratique.

II. Présentation du BUS CAN :

1. Introduction :

Le Controller Area Network (CAN) est un système de communication en série asynchrone développé par Bosch GmbH dans les années 1980. Initialement conçu pour l'industrie automobile, son objectif était de remplacer les faisceaux de câbles complexes par un simple bus à deux fils. Le CAN permet une robustesse aux interférences EMI et la capacité d'auto-vérification et de correction des erreurs. Il fonctionne comme un bus multi-maître connectant diverses unités de contrôle appelées nœuds. Tous les nœuds connectés au bus peuvent simultanément recevoir chaque message, offrant ainsi une communication efficace sans nécessité d'un hôte.

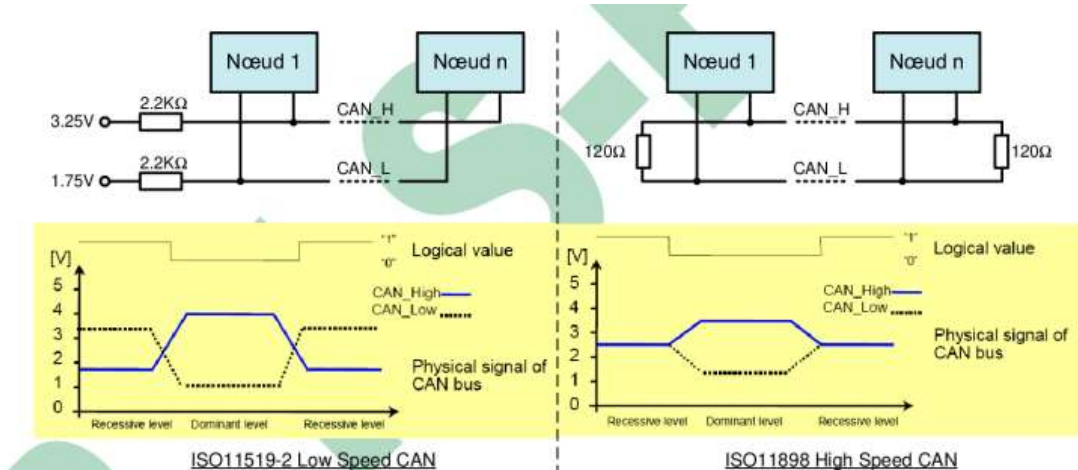
2. Origine :

Dans les années 80, la croissance des organes électroniques dans l'industrie automobile a incité à réduire le nombre de lignes dédiées entre les organes. La collaboration entre l'Université de Karlsruhe et Bosch a conduit à la création du bus CAN, permettant le multiplexage des réseaux des véhicules. Actuellement utilisé dans diverses industries, il a été standardisé par l'ISO dans les années 1990 après sa présentation avec Intel en 1985. En 1992, plusieurs entreprises ont formé le CAN in Automation, une association visant à promouvoir le CAN.

3. Principes Généraux de Communication :

- **Topologie et Objets (Nœuds) :** Le CAN utilise une topologie de bus avec la notion d'objets ou de nœuds, permettant théoriquement jusqu'à 2032 objets avec la version CAN 2.0A pour un maximum de 32 nœuds.
- **Fonctionnement Multi-Maîtres :** Lorsque le bus est libre, chaque nœud peut décider d'envoyer un message, et seul le message de plus haute priorité prend possession du bus.
- **Médium de Transmission :** Le médium est un câble à paires torsadées à 120 ohms, blindé ou non, avec une transmission en mode différentiel.
- **Codage des Signaux :** Le codage NRZ binaire est utilisé, avec deux niveaux logiques reconnus :

- ✓ Bit Récessif '1' : Tension différentielle inférieure à -0,3V



Vitesse	Jusqu'à 125 kbps						Jusqu'à 1 Mbps					
Longueur Bus	1 km / 40 Kbps						40 m / 1 Mbps					
Nbre de nœuds	Maximum 20						Maximum 30					
Topologie	Récessif			Dominant			Récessif			Dominant		
	Min	Nom	Max	Min	Nom	Max	Min	Nom	Max	Min	Nom	Max
CAN_High (V)	1.6	1.75	1.9	3.85	4	5	2	2.5	3	2.75	3.5	4.5
CAN_Low (V)	3.1	3.25	3.4	0	1	1.15	2	2.5	3	0.5	1.5	2.25
CAN_H - CAN_L	-0.3	-1.5	-	0.3	3	-	-0.5	0	0.05	1.5	2	3

Note : Pour résoudre le problème de fiabilité du codage NRZ (succession de bits identiques), on utilise la technique du **Bit Stuffing** qui impose au transmetteur d'ajouter automatiquement **un bit de valeur opposée** lorsqu'il détecte 5 bits consécutifs dans les valeurs à transmettre :



Figure 1: Principe généraux de communication

- ✓ Bit Dominant '0' : Tension différentielle comprise entre 0,3V et 3V.

4.Principe de Fonctionnement :

a- Routage des Informations :

Le bus CAN fonctionne sur le concept de diffusion générale d'information, où chaque information est diffusée sur le bus sous la forme d'un message (trame de bits) de format défini mais de longueur variable. Lorsque le bus est libre, n'importe quel nœud du réseau peut émettre un nouveau message. Chaque message est doté d'un identificateur qui ne spécifie pas la destination du message, mais plutôt la signification des données qu'il transporte. Cet identificateur sert également à déterminer la priorité des messages en cas de conflit. Chaque station du réseau filtre

les trames en utilisant des masques, déterminant ainsi les messages qui lui sont destinés. Les nœuds peuvent détecter des erreurs sur des messages qui ne leur sont pas destinés et en informer les autres nœuds. De plus, de nouveaux nœuds peuvent être ajoutés au réseau sans nécessiter de modifications au niveau logiciel ou matériel.

b- arbitrage :

Afin de garantir une transmission rapide et efficace des données en temps réel, le Bus CAN met en place des mécanismes spécifiques pour traiter les informations de manière urgente. Voici les points clés de cette gestion de priorité et d'arbitrage :

1. Voie Physique de Transmission :

- ✓ Une voie physique de transmission atteignant jusqu'à 1 Mbps est utilisée, assurant une communication rapide et efficace sur le bus CAN.

2. Mécanisme d'Allocation de Bus Efficace :

- ✓ -Un mécanisme d'allocation de bus efficace est nécessaire pour traiter les conflits sans perte de temps ni d'informations. Cela garantit une utilisation optimale du bus pour une communication fluide.

3. Urgence des Informations :

- ✓ L'urgence des informations échangées sur le bus peut varier. Par exemple, une valeur variante rapidement, comme l'état d'un capteur ou l'asservissement de la position d'un moteur, doit avoir la priorité par rapport à des valeurs variant lentement, telles que la température d'un moteur.

4. Méthode d'Accès CSMA-CD/AMP :

- ✓ Pour déterminer la priorité d'un message, le Bus CAN utilise la méthode d'accès CSMA-CD/AMP (Carrier Sense Multiple Access with Collision Detection and Arbitration Message Priority).
- ✓ L'identificateur de chaque message, un mot de 11 bits (format standard CAN 2.0A) ou 29 bits (format étendu CAN 2.0B), est crucial pour cette détermination.

5. Arbitrage Bit à Bit :

- ✓ En cas de plusieurs nœuds tentant d'émettre un message sur un bus libre, un arbitrage bit à bit (non destructif) est effectué tout au long du contenu de l'identificateur.
- ✓ Le premier nœud présentant un bit récessif alors qu'au moins un autre présente un bit dominant perd l'arbitrage.

6. Réaction en Cas de Perte d'Arbitrage :

- ✓ Lorsqu'un nœud perd l'arbitrage, il devient automatiquement un récepteur du message en cours de transmission.
- ✓ Il tentera de retransmettre lorsque le bus sera à nouveau libre.

✚ **En résumé, la combinaison d'une voie de transmission rapide, d'un mécanisme d'allocation de bus efficace et de l'utilisation de l'identificateur pour l'arbitrage permet au Bus CAN de traiter efficacement les informations en temps réel tout en respectant les priorités définies.**

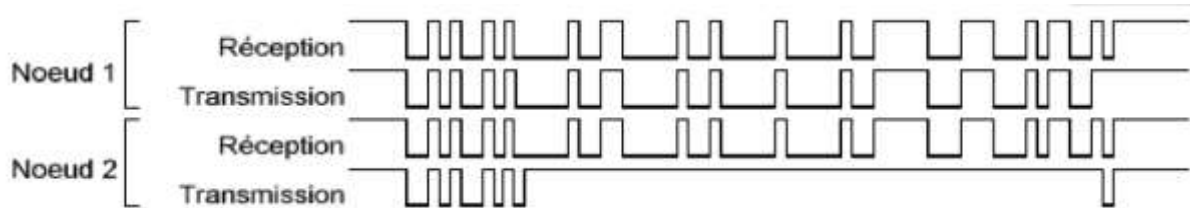


Figure 2: Arbitrage

5. Trame de données :

Une trame de données se décompose en 7 champs différents :

- le début de trame SOF : 1 bit dominant.
- le champ d'arbitrage : 12 ou 30 bits.
- le champ de contrôle : 6 bits.
- le champ de données : 0 à 64 bits.
- le champ de CRC : 16 bits.
- le champ d'acquittement (Acknowledge) : 2 bits.
- le champ de fin de trame (End of Frame) : 7 bits récessifs.

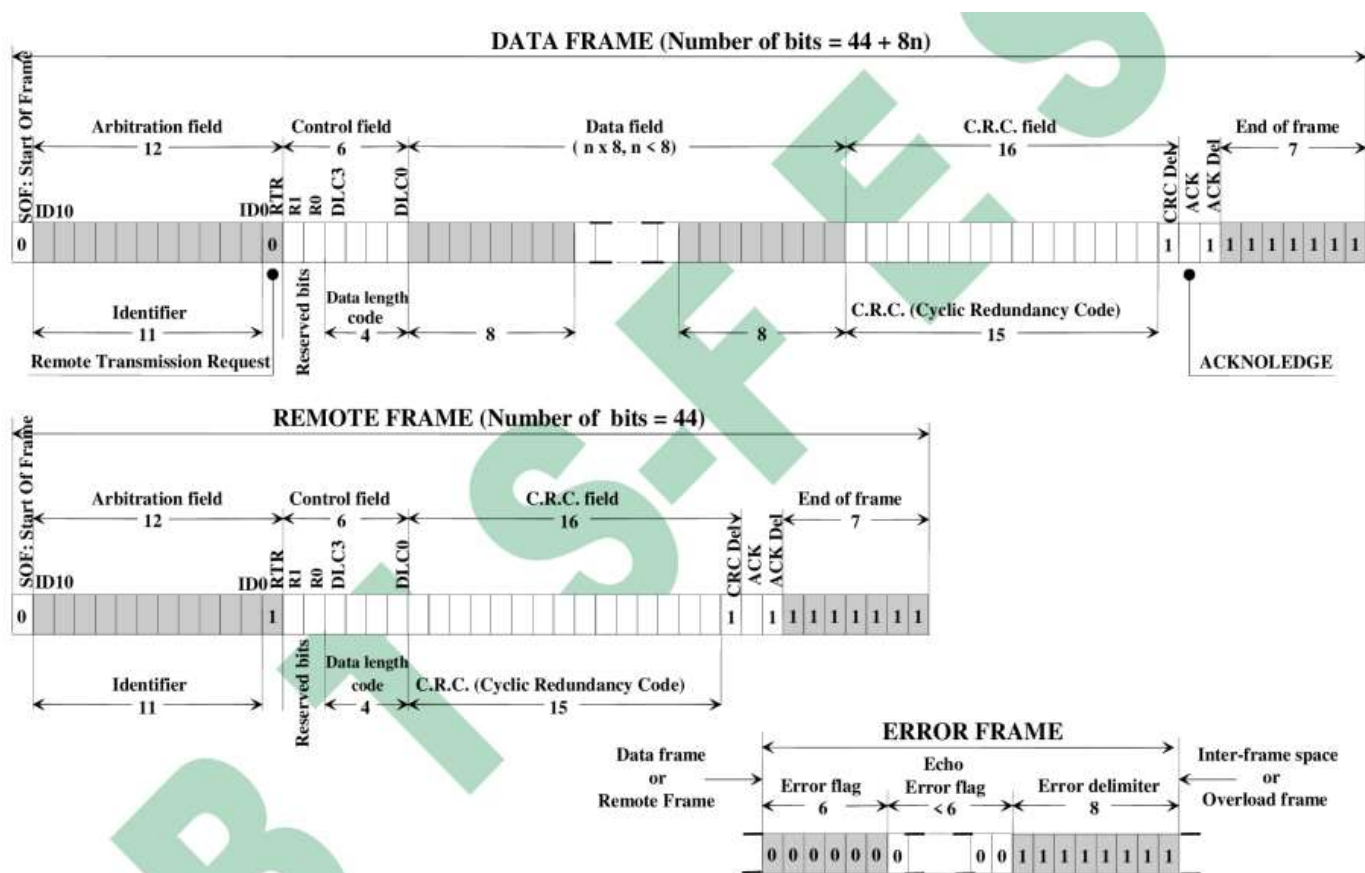


Figure 3: Trame de donnée

CHAPITRE II.

ENVIRONNEMENT MATÉRIEL

Pour la réalisation de notre projet nous avons utilisée plusieurs composant :

I. Le microcontrôleur :

Un microcontrôleur est un petit ordinateur sur une seule puce intégré dans un circuit intégré. Il est conçu pour être utilisé dans des systèmes embarqués tels que des robots, des appareils électroniques, des équipements médicaux, et bien d'autres. Les microcontrôleurs sont souvent utilisés pour contrôler des fonctions spécifiques dans ces appareils, comme par exemple la gestion de capteurs, l'affichage de l'information, la régulation de la température, et bien d'autres tâches. Ils sont largement utilisés dans l'industrie et dans la vie quotidienne en raison de leur taille compacte, de leur efficacité énergétique et de leur coût relativement faible.

1. Le choix d'un microcontrôleur :

Pour notre projet de commande d'une interface Bus CAN pour les rétroviseurs de voiture, le choix du microcontrôleur a été déterminant. Nous avons opté pour un microcontrôleur compatible avec le Bus CAN, garantissant ainsi une communication fluide et efficace. La puissance de traitement et la capacité mémoire du microcontrôleur ont été évaluées en fonction des besoins spécifiques du projet, assurant ainsi des performances optimales. De plus, nous avons pris en compte la disponibilité des périphériques intégrés nécessaires, ainsi que la présence de bibliothèques logicielles et de support, facilitant le développement et l'implémentation du système. Enfin, la disponibilité et le coût du microcontrôleur ont également été des facteurs déterminants dans notre choix, garantissant une approche pratique et économique pour la réalisation du projet.

2. Arduino Uno :

L'Arduino Uno est une carte microcontrôleur à usage général, largement utilisée pour le prototypage et la réalisation de projets électroniques. Elle offre une plateforme conviviale et adaptable pour le développement de diverses applications embarquées.



Figure 4:Arduino Uno

Il se caractérise par :

- 1. Microcontrôleur :** Arduino Uno est basé sur le microcontrôleur ATmega328P de chez Atmel, cadencé à 16 MHz.
- 2. Mémoire Flash :** Il dispose de 32 kilooctets de mémoire Flash, utilisée pour le stockage du programme.
- 3. SRAM :** L'Arduino Uno est équipé de 2 kilooctets de SRAM, utilisés pour le stockage temporaire des données pendant l'exécution du programme.
- 4. EEPROM :** La carte possède 1 kilooctet d'EEPROM, qui permet de stocker des données de manière non volatile, même lorsque l'alimentation est coupée.
- 5. Broches d'Entrées/Sorties Numériques :** Il dispose de 14 broches numériques pouvant être utilisées comme entrées ou sorties.
- 6. Broches d'Entrées Analogiques :** L'Arduino Uno comporte 6 broches analogiques pour mesurer des signaux analogiques.
- 7. Interfaces de Communication :** Il est équipé d'une interface USB pour la programmation et la communication série avec l'ordinateur. De plus, il possède des broches TX/RX pour la communication série avec d'autres dispositifs.
- 8. Connecteur d'Alimentation :** L'Arduino Uno peut être alimenté via une source USB, une batterie externe ou un adaptateur secteur. Il fonctionne sous une tension de 5 volts.
- 9. LEDs d'État :** La carte inclut une LED d'alimentation, une LED de communication série, et 13 LEDs associées aux broches numériques pour faciliter le débogage.
- 10. Interrupteurs :** Il comporte un interrupteur de réinitialisation ainsi qu'un interrupteur d'alimentation.
- 11. Horloge :** L'Arduino Uno intègre un oscillateur à cristal de 16 MHz pour assurer la synchronisation des opérations.
- 12. Dimensions :** Physiquement, la carte Arduino Uno a des dimensions compactes, ce qui la rend facile à intégrer dans différents projets.

✚ Ces caractéristiques font de l'Arduino Uno un choix polyvalent pour les projets électroniques, qu'ils soient destinés aux débutants ou aux utilisateurs plus avancés.

3. Noms et fonctions des broches d'un Arduino Uno :

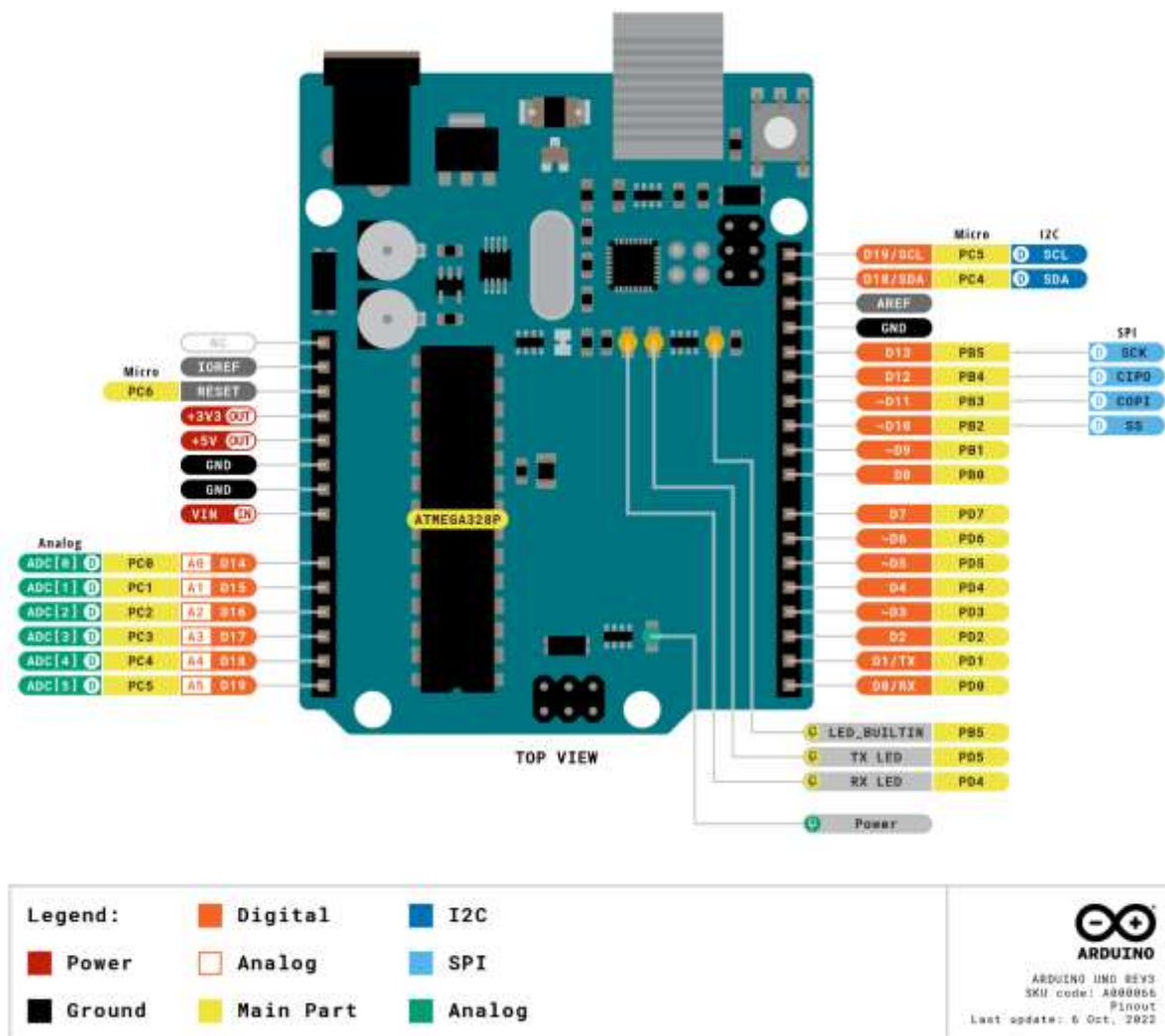


Figure 5: Datasheet d'un Arduino Uno

II. AFFICHEUR LCD 20*4 :



Figure 6: afficheur LCD 20*4

Un afficheur LCD 20x4 est un écran à cristaux liquides qui peut afficher 20 caractères par ligne sur 4 lignes, offrant ainsi une surface d'affichage plus grande par rapport aux écrans LCD plus petits.

Voici quelques caractéristiques typiques d'un afficheur LCD 20x4 :

- 1. Dimensions :** Un afficheur LCD 20x4 a quatre lignes, chacune capable d'afficher 20 caractères. Cela donne une surface d'affichage de 20 caractères par ligne sur 4 lignes.
- 2. Rétroéclairage :** La plupart des afficheurs LCD 20x4 sont équipés d'un rétroéclairage LED qui permet de lire les informations affichées même dans des conditions de faible luminosité.
- 3. Interface :** Ils utilisent généralement une interface parallèle, ce qui signifie qu'ils nécessitent plusieurs broches pour être connectés à un microcontrôleur (par exemple, à travers des broches de données et des broches de contrôle).
- 4. Contrôleur :** Les afficheurs LCD 20x4 utilisent souvent des contrôleurs compatibles HD44780 ou équivalents. Ces contrôleurs facilitent la communication avec un microcontrôleur.
- 5. Alimentation :** Ils fonctionnent généralement sous une tension d'alimentation de 5 volts.
- 6. Caractères :** Ces afficheurs peuvent généralement afficher des caractères alphanumériques ainsi que des symboles spéciaux.
- 7. Réglage du Contraste :** Certains modèles peuvent inclure un potentiomètre pour régler le contraste de l'affichage.
- 8. Utilisation Courante :** Ces écrans sont couramment utilisés dans des projets électroniques pour afficher des données telles que des mesures de capteurs, des informations sur l'état d'un système, des messages ou des menus.
- 9. Commandes Programmables :** Ils peuvent être programmés pour afficher différentes informations en fonction des besoins du projet.

- ✚ L'utilisation de cet afficheur LCD 20x4 dans un projet peut ajouter une interface utilisateur visuelle, offrant la possibilité d'afficher des informations de manière claire et lisible.

1. Connection d'un afficheur avec Arduino Uno :

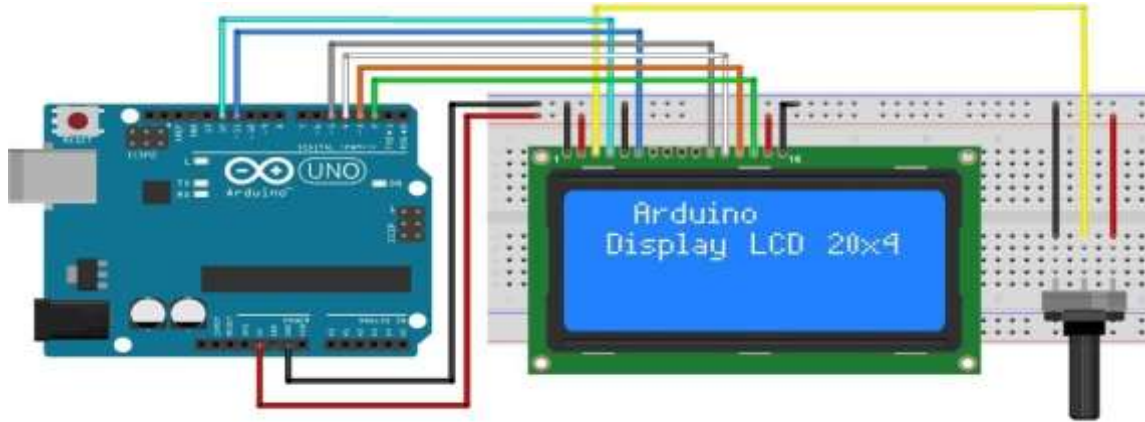


Figure 7: Connection d'un afficheur avec Arduino Uno

La connexion d'un afficheur LCD 20x4 avec Arduino implique l'utilisation d'une interface parallèle.

➤ Les Étapes de connexion :

1. Connectez l'afficheur LCD à l'Arduino :

- **Broches d'alimentation** : Connectez le fil rouge (+5V) de l'afficheur à la broche 5V de l'Arduino, et le fil noir (GND) à la broche GND.
- **Broches de données** : Connectez les broches de données (D4 à D7) de l'afficheur aux broches numériques correspondantes de l'Arduino (par exemple, D4 à D7).
- **Broches de contrôle** : Connectez les broches RS, RW et E de l'afficheur à des broches numériques de l'Arduino. Généralement, RS est connecté à D8, RW à la masse (GND), et E à D9.

2. Alimentation :

- Assurez-vous que l'Arduino est alimenté. Vous pouvez le faire via le port USB ou une autre source d'alimentation.

2. Noms et fonctions des broches des écrans LCD de caractères :

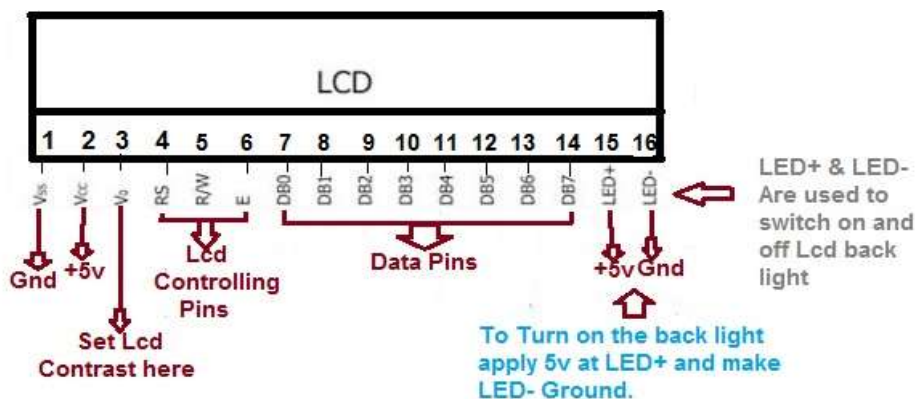


Figure 8: datasheet d'un afficheur LCD

III. Transmetteur \ Récepteur BUSCAN MODULE (MCP2515) :



Figure 9: MCP2515 CAN Module

✚ Le MCP2515 est un circuit intégré couramment utilisé comme transmetteur/récepteur pour les communications CAN.

1.Fonction : Le MCP2515 est un contrôleur CAN (Contrôleur d'Accès au Réseau) qui agit en tant que transmetteur et récepteur pour les communications sur le bus CAN.

2.Interface SPI : Il utilise une interface série périphérique (SPI) pour la communication avec un microcontrôleur, ce qui facilite l'intégration avec des plates-formes comme Arduino.

3.Fonction de Transmetteur : En tant que transmetteur, le MCP2515 peut prendre les données d'un microcontrôleur via l'interface SPI et les envoyer sur le bus CAN.

4.Fonction de Récepteur : En tant que récepteur, il peut écouter les données sur le bus CAN et les transmettre au microcontrôleur via l'interface SPI.

5.Vitesse de Communication : Il prend en charge plusieurs vitesses de communication CAN, permettant de s'adapter aux différents débits de transmission utilisés dans les systèmes embarqués.

6.Tampons de Réception et de Transmission : Le MCP2515 dispose de tampons dédiés à la réception et à la transmission des données, ce qui lui permet de traiter efficacement les messages sur le bus CAN.

7.Filtrage et Masquage : Il offre des fonctionnalités de filtrage et de masquage, permettant de configurer des filtres pour sélectionner les messages spécifiques que le module doit recevoir.

8.Tolérance aux Erreurs : Le MCP2515 est conçu pour gérer les erreurs de communication sur le bus CAN, améliorant ainsi la fiabilité de la transmission des données.

9.Compatibilité avec les Modes Étendus : Il prend en charge les modes standard et étendu du protocole CAN, permettant ainsi la communication avec différents types de dispositifs CAN.

10.Rétroaction d'Erreur : Le MCP2515 peut générer des signaux d'interruption en cas d'erreur de communication, fournissant des informations au microcontrôleur sur les problèmes éventuels.

- ✚ En résumé, le MCP2515 est un composant essentiel dans la mise en œuvre de la communication CAN dans de nombreux projets électroniques, en permettant au microcontrôleur de transmettre et de recevoir des données sur un bus CAN de manière efficace et fiable.

1. Noms des broches du BUSCAN (MCP2515) :

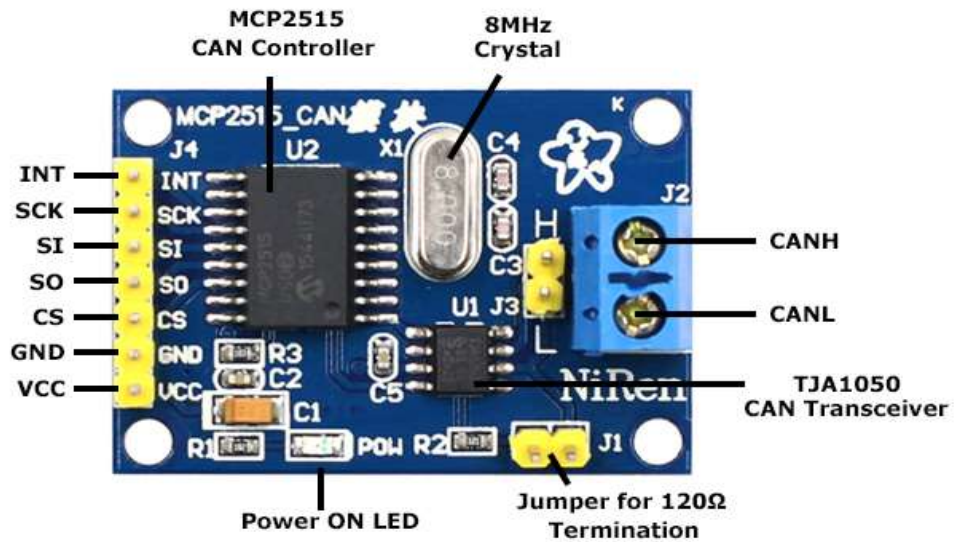


Figure 10: Datasheet d'un BUSCAN (MCP2515)

2. description des broches :

TABLE 1-1: PINOUT DESCRIPTION

Name	PDIP/ SOIC Pin #	TSSOP Pin #	QFN Pin #	I/O/P Type	Description	Alternate Pin Function
TXCAN	1	1	19	O	Transmit output pin to CAN bus	—
RXCAN	2	2	20	I	Receive input pin from CAN bus	—
CLKOUT	3	3	1	O	Clock output pin with programmable prescaler	Start-of-Frame signal
TX0RTS	4	4	2	I	Transmit buffer TXB0 Request-to-Send; 100 k Ω internal pull-up to V _{DD}	General purpose digital input, 100 k Ω internal pull-up to V _{DD}
TX1RTS	5	5	3	I	Transmit buffer TXB1 Request-to-Send; 100 k Ω internal pull-up to V _{DD}	General purpose digital input, 100 k Ω internal pull-up to V _{DD}
TX2RTS	6	7	5	I	Transmit buffer TXB2 Request-to-Send; 100 k Ω internal pull-up to V _{DD}	General purpose digital input, 100 k Ω internal pull-up to V _{DD}
OSC2	7	8	6	O	Oscillator output	—
OSC1	8	9	7	I	Oscillator input	External clock input
V _{SS}	9	10	8	P	Ground reference for logic and I/O pins	—
RX1BF	10	11	9	O	Receive buffer RXB1 interrupt pin or general purpose digital output	General purpose digital output
RX0BF	11	12	10	O	Receive buffer RXB0 interrupt pin or general purpose digital output	General purpose digital output
INT	12	13	11	O	Interrupt output pin	—
SCK	13	14	12	I	Clock input pin for SPI interface	—
SI	14	16	14	I	Data input pin for SPI interface	—
SO	15	17	15	O	Data output pin for SPI interface	—
CS	16	18	16	I	Chip select input pin for SPI interface	—
RESET	17	19	17	I	Active-low device Reset input	—
V _{DD}	18	20	18	P	Positive supply for logic and I/O pins	—
NC	—	6,15	4,13	—	No internal connection	—
EP	—	—	21	—	Exposed Thermal Pad, connect to V _{SS}	—

Legend: I = Input; O = Output; P = Power

Tableau 1: description des broches

3. Diagramme :

BUFFERS

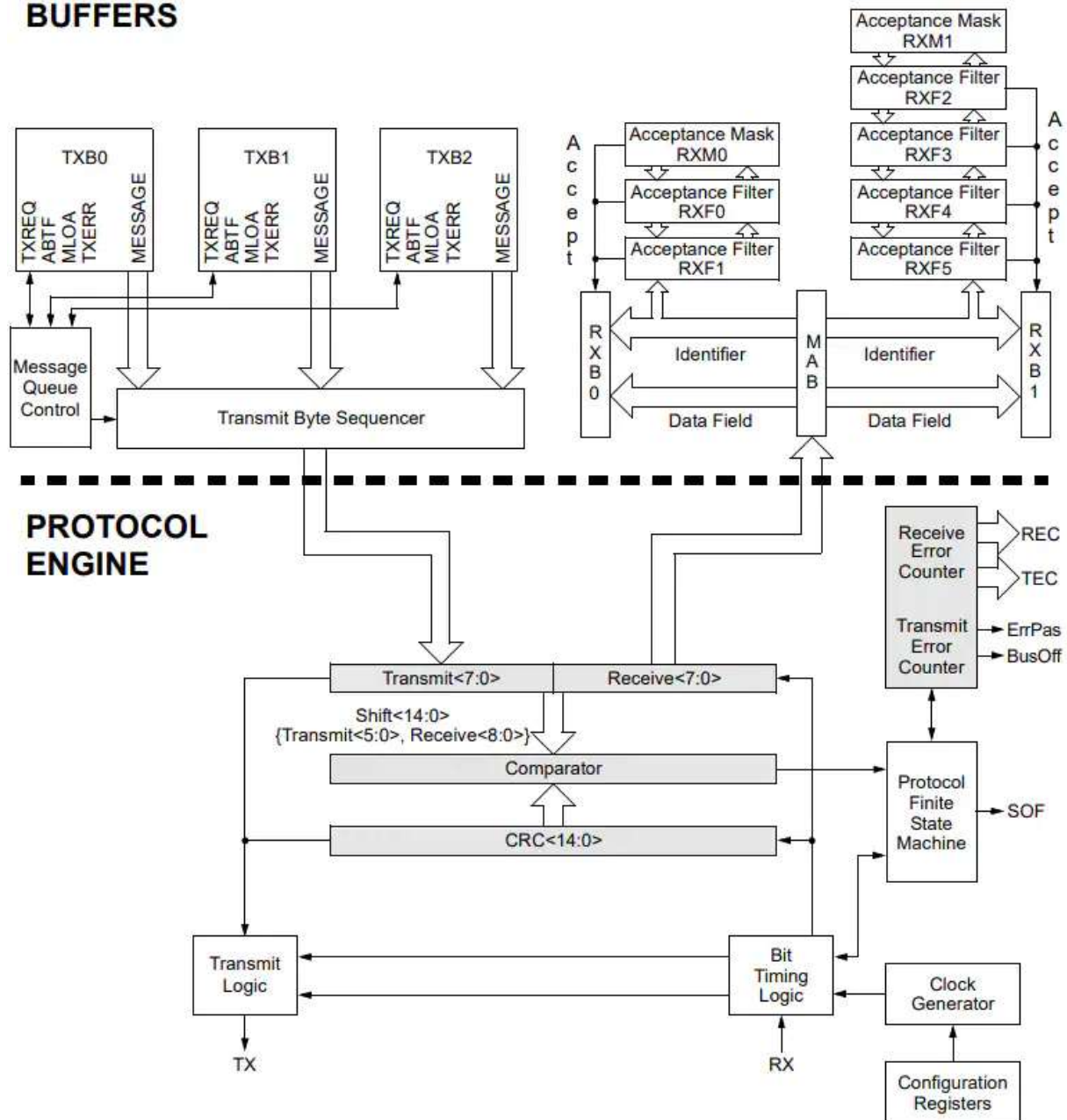


Figure 11: Diagramme

IV. interactive 3 positions rotary switch :

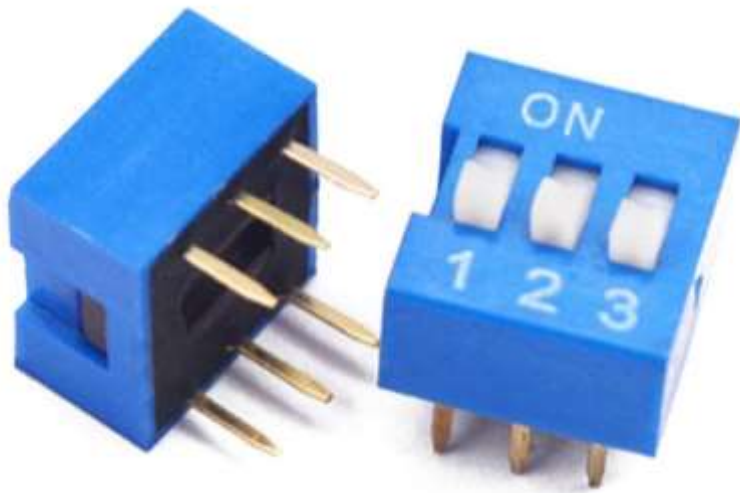


Figure 12: interactive 3 positions rotary switch

Un interrupteur à glissière est un dispositif électrique qui utilise une partie mobile, appelée curseur, pour créer, interrompre ou dévier un courant électrique en la déplaçant entre différentes positions prédéfinies. Ces interrupteurs sont souvent utilisés dans les circuits électroniques pour permettre à l'utilisateur de choisir entre différentes configurations, régler des paramètres ou activer/désactiver des fonctions spécifiques en faisant glisser la partie mobile le long d'une piste conductrice.

Voici quelques détails sur ce type d'interrupteur :

- 1. 3 Positions :** Comme indiqué, l'interrupteur à glissière à 3 positions peut être déplacé dans trois positions distinctes. Chaque position peut correspondre à une fonction spécifique ou à un état dans un circuit électrique.
- 2. À Glissière :** Ce type d'interrupteur utilise un mécanisme de glissement pour passer d'une position à l'autre. Vous pouvez déplacer manuellement la partie coulissante de l'interrupteur vers le haut, le bas ou le centre pour sélectionner la position souhaitée.
- 3. Interactif :** L'utilisateur peut interagir avec l'interrupteur en déplaçant la partie coulissante vers la position désirée.
- 4. Connectivité :** Comme tout interrupteur, l'interrupteur à glissière aurait des contacts électriques internes qui changent d'état en fonction de la position de la glissière.
- 5. Applications :** Les interrupteurs à glissière à 3 positions peuvent être utilisés dans diverses applications électroniques où le choix entre trois états différents est nécessaire.
- 6. Montage :** Ils sont disponibles dans différentes configurations de montage, y compris le montage en surface, le montage encastré ou le montage sur panneau.
- 7. Rétroaction :** Certains modèles peuvent offrir une sensation tactile lors du déplacement de la glissière, permettant à l'utilisateur de sentir clairement le passage d'une position à l'autre.

V. Bouton-poussoir :



Figure 13: Bouton poussoir

Un bouton-poussoir est un type d'interrupteur électrique qui est conçu pour être enfoncé (poussé) pour activer une fonction et relâché pour la désactiver. Il s'agit d'un interrupteur momentané, ce qui signifie qu'il n'a pas de position verrouillée en position « activée » ou « désactivée ». Lorsque vous appuyez sur le bouton, il établit une connexion électrique, et dès que vous relâchez le bouton, la connexion est interrompue.

Voici quelques caractéristiques clés d'un bouton-poussoir :

- 1. Momentané :** Le bouton n'a pas de position verrouillée. Son état dépend de la pression exercée sur le bouton.
- 2. Action :** L'action du bouton consiste à appuyer sur le bouton pour effectuer une action momentanée.
- 3. Utilisations courantes :** Les boutons-poussoirs sont fréquemment utilisés dans les dispositifs électroniques, les tableaux de bord de voitures, les télécommandes, etc.
- 4. Connectivité :** Lorsqu'il est enfoncé, le bouton établit une connexion électrique, et lorsque vous le relâchez, la connexion est interrompue.
- 5. Rétroaction tactile :** Certains boutons-poussoirs offrent une sensation tactile ou un « clic » lorsqu'ils sont enfoncés, ce qui peut fournir une rétroaction à l'utilisateur.
- 6. Variétés :** Il existe différentes tailles et formes de boutons-poussoirs pour s'adapter à diverses applications.
 - ✚ Les boutons-poussoirs peuvent être utilisés pour diverses fonctions, notamment comme interrupteurs d'alimentation, déclencheurs d'événements dans des circuits électroniques, ou dans des applications où une action momentanée est nécessaire.

VI. Module L298 pour commande de moteur DC :

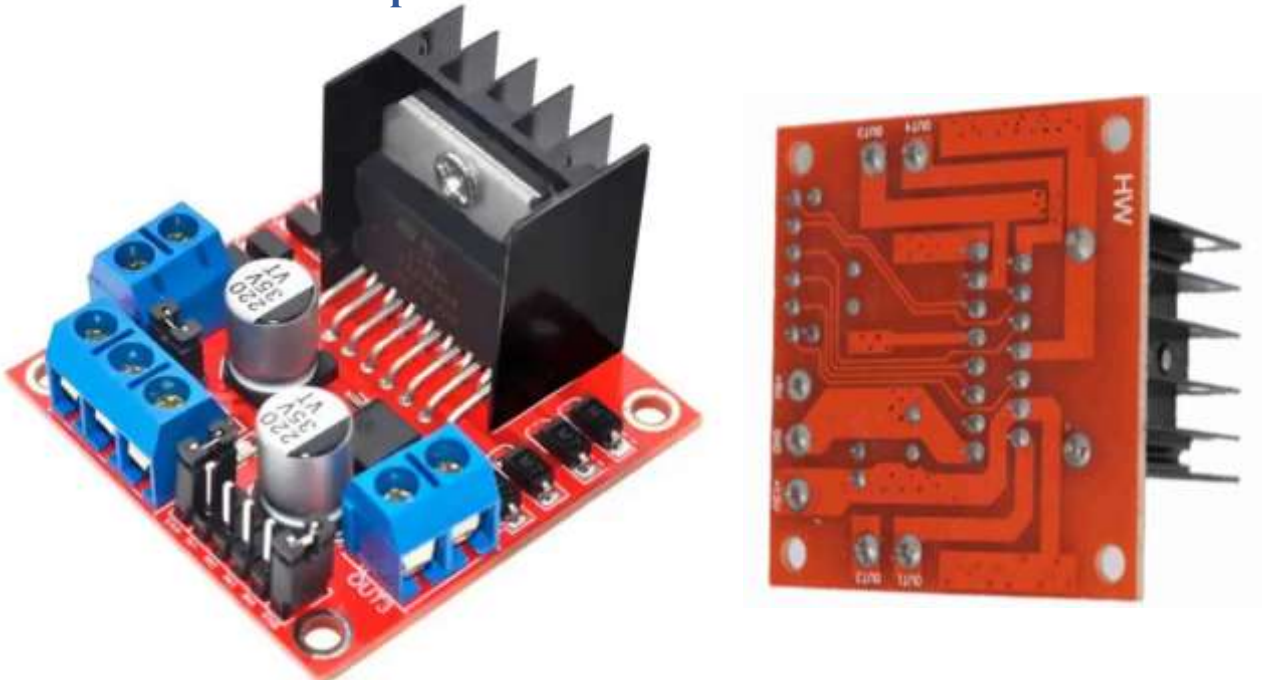


Figure 14: Module L298 pour commande de moteur DC

Le module L298 est un circuit intégré populaire utilisé pour la commande de moteurs à courant continu (DC). Il est couramment utilisé dans des applications telles que la robotique, les projets de bricolage, les véhicules électriques et d'autres systèmes nécessitant la commande de moteurs DC. Voici quelques caractéristiques et informations générales sur le module L298 :

- 1. Pont en H :** Le module L298 est souvent appelé un « pont en H ». C'est une configuration de circuits utilisée pour inverser la polarité d'un moteur DC, permettant ainsi le contrôle de la direction du moteur.
- 2. Contrôle bidirectionnel :** Le L298 permet le contrôle bidirectionnel d'un moteur DC, ce qui signifie qu'il peut faire tourner le moteur dans les deux sens (avant et arrière).
- 3. Courant élevé :** Il peut gérer des courants relativement élevés, ce qui en fait un choix approprié pour des moteurs DC de puissance modérée à élever.
- 4. Tension d'alimentation :** Le module L298 peut fonctionner avec une large gamme de tensions d'alimentation, généralement dans la plage de 5 à 35 volts.
- 5. Commande simple :** Il peut être commandé à l'aide de signaux de commande simples, ce qui facilite son intégration dans des projets électroniques.
- 6. Commande PWM :** Il prend en charge la modulation de largeur d'impulsion (PWM), ce qui permet de contrôler la vitesse du moteur en ajustant la fréquence des impulsions.

7. Protection thermique : Certains modules L298 sont équipés de protections thermiques intégrées pour éviter la surchauffe du circuit.

8. Applications : Le module L298 est largement utilisé dans la conception de robots, de véhicules téléguidés, de systèmes de positionnement, et d'autres projets nécessitant la commande précise de moteurs DC.

- ✚ Lors de l'utilisation du module L298, il est important de se référer aux spécifications du fabricant et de suivre les schémas de câblage recommandés pour assurer un fonctionnement correct et sûr.

1. Les broches de module moteur :

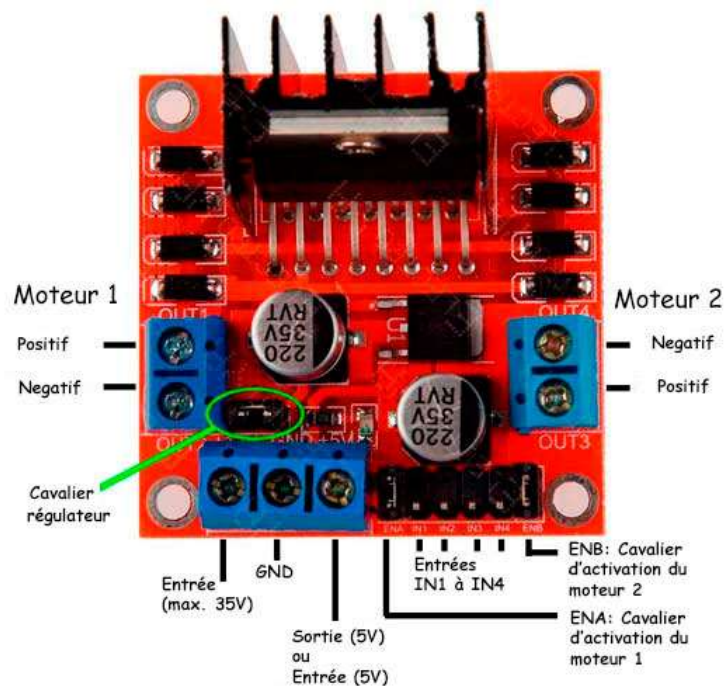


Figure 15: datasheet d'un module L298

2. Connection d'un hacheur avec Arduino :

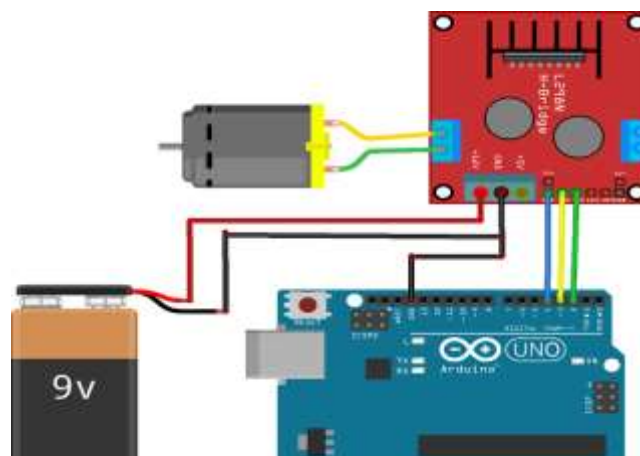


Figure 16: connexion d'un hacheur avec Arduino

VII. Moteur DC :



Figure 17: Moteur DC

Un moteur à courant continu est un type de moteur électrique qui fonctionne en convertissant l'énergie électrique continue en mouvement mécanique. Voici quelques points clés à connaître sur les moteurs DC :

- 1. Principe de fonctionnement :** Les moteurs DC fonctionnent sur le principe de l'électromagnétisme. Ils utilisent le champ magnétique créé par le courant électrique circulant dans une bobine (ou un enroulement) pour générer un couple, provoquant ainsi la rotation de l'arbre du moteur.
- 2. Courant continu :** Ils fonctionnent avec une alimentation en courant continu, où le courant électrique circule dans une seule direction.
- 3. Deux types principaux :** Il existe deux principaux types de moteurs DC – les moteurs à courant continu sans balais (brushless DC motors) et les moteurs à courant continu avec balais (brushed DC motors). Les moteurs sans balais sont plus modernes, plus efficaces et nécessitent électroniquement plus de contrôle que les moteurs avec balais.
- 4. Applications variées :** Les moteurs DC sont largement utilisés dans diverses applications, notamment dans l'industrie automobile, les appareils électroménagers, les jouets, les outils électriques, les systèmes de ventilation, les robots, etc.
- 5. Contrôle de vitesse :** La vitesse d'un moteur DC peut être contrôlée en ajustant la tension appliquée ou en utilisant des techniques de modulation de largeur d'impulsion (PWM).
- 6. Couple et puissance :** Le couple d'un moteur DC dépend du courant circulant dans la bobine et du nombre de tours par minute (tr/min) de l'arbre du moteur. La puissance dépend du couple et de la vitesse de rotation.
- 7. Inversion de sens de rotation :** Le sens de rotation d'un moteur DC peut être inversé en inversant la polarité de l'alimentation.

8. Entretien pour les moteurs avec balais : Les moteurs avec balais nécessitent un entretien périodique pour remplacer les balais, tandis que les moteurs sans balais sont sans entretien.

9. Contrôle électronique : Les moteurs DC, en particulier les moteurs sans balais, peuvent être contrôlés électroniquement pour une performance optimale, une régulation de vitesse précise et une efficacité énergétique.

10. Tailles et puissances variées : Les moteurs DC sont disponibles dans une gamme de tailles et de puissances pour répondre à divers besoins d'application.

- ✚ Les moteurs à courant continu sont des composants essentiels dans de nombreuses technologies modernes en raison de leur polyvalence, de leur facilité de contrôle et de leur large éventail d'applications.

VIII. Potentiomètre :



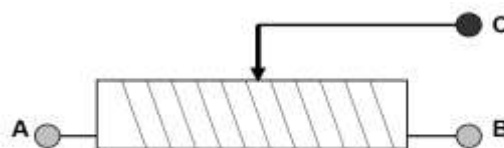
Figure 18: Potentiomètre

1. Principe de fonctionnement :

Obtenir une tension de sortie variable à l'aide d'un pont diviseur de tension équipé d'une résistance variable montée en potentiomètre en fonction de la tension d'entrée.

2. Constitution :

La résistance variable est un composant très utilisé en électronique. Elle est composée d'un résistor fixe (A et B) où on a ajouté un curseur (C) qui vient faire contact avec un conducteur résistif non isolé appelée piste résistive. En déplaçant le curseur sur le conducteur résistif, on fait varier la valeur de la résistance entre les bornes A et C ou entre B et C.



3. La résistance variable possède trois caractéristiques :

- La valeur maximale de la résistance.
- L'intensité maximale.
- La puissance admissible.

N'oublions pas que la résistance variable, quel que soit son utilisation est régie par la loi d'ohm et que tout courant qui traverse cette résistance produit de la chaleur.

CHAPITRE III.

APPLICATIONS ET RÉALISATION

I. Description du système :

Le système élaboré repose sur une architecture organisée en deux parties distinctes, chacune gérant des aspects spécifiques de la commande des rétroviseurs d'une voiture. Arduino 1 est dédié à l'interaction avec l'utilisateur, étant équipé de quatre boutons-poussoirs (UP, DOWN, RIGHT, LEFT) pour des commandes manuelles. Il est également connecté à un afficheur LCD, fournissant ainsi une interface visuelle pour afficher des informations sur l'état du système. Arduino 1 communique en permanence avec Arduino 2 via le MCP2515, assurant un échange bidirectionnel de données.

D'autre part, Arduino 2 se concentre sur le contrôle des moteurs des rétroviseurs. Il est connecté au contrôleur de moteur L298, qui gère les mouvements des moteurs situés dans le rétroviseur. Les commandes de positionnement des rétroviseurs sont reçues de manière précise de la part d'Arduino 1 via la communication sur le bus CAN. En réponse à ces commandes, Arduino 2 ajuste les moteurs pour positionner les rétroviseurs conformément aux instructions reçues.

- ✚ Cette organisation du système permet une séparation claire des tâches entre Arduino 1 et Arduino 2. Arduino 1 agit comme une interface utilisateur, facilitant les commandes manuelles et fournissant des retours visuels, tandis qu'Arduino 2 se charge du contrôle précis des moteurs en réponse aux instructions reçues. Ensemble, ces deux composants collaborent de manière synergique pour offrir une solution complète et efficace pour la commande synchronisée des rétroviseurs d'une voiture.

1. Description du microcontrôleur ARD1 :

ARD1, désignant Arduino 1 dans le contexte du projet, est le microcontrôleur central jouant un rôle essentiel dans la gestion de l'interface utilisateur. Avec son utilisation, nous avons choisi Arduino Uno en raison de sa simplicité d'utilisation, de sa polyvalence, et de sa compatibilité avec le Bus CAN.

a) *Ça capacité de communication avec le BUS CAN :*

La capacité de communication d'ARD1 avec le Bus CAN est rendue possible par l'incorporation du module MCP2515. Ce module agit comme un transmetteur/récepteur CAN, permettant à ARD1 d'échanger des données de manière bidirectionnelle sur le Bus CAN. Le MCP2515 assure la conversion des signaux entre le microcontrôleur et le protocole de communication du Bus CAN. Cette capacité de communication avec le Bus CAN offre à ARD1 la possibilité d'envoyer des commandes de rétroviseurs et de recevoir des informations sur l'état du système, contribuant ainsi à la synchronisation précise des rétroviseurs de la voiture.

b) *Gestion des boutons de commande des rétroviseurs :*

La gestion des boutons de commande des rétroviseurs par ARD1 est une fonction cruciale dans le système. Équipé de quatre boutons-poussoirs (UP, DOWN, RIGHT, LEFT), ARD1 interprète les commandes manuelles de l'utilisateur pour ajuster la position des rétroviseurs de la voiture. Chaque bouton est associé à une direction spécifique, permettant des commandes précises pour déplacer les rétroviseurs vers le haut, vers le bas, vers la droite ou vers la gauche. La détection des pressions sur les boutons est gérée par ARD1, qui traduit ces commandes en signaux appropriés à envoyer via le Bus CAN. Cette gestion des boutons

assure une interface conviviale et réactive, offrant à l'utilisateur un contrôle direct sur la position des rétroviseurs.

c) Traitement des commandes et contrôle des rétroviseurs :

Le traitement des commandes et le contrôle des rétroviseurs par ARD1 sont des éléments essentiels du fonctionnement du système. Lorsque l'utilisateur actionne les boutons-poussoirs associés (UP, DOWN, RIGHT, LEFT), ARD1 interprète ces commandes pour générer des signaux adaptés. Ces signaux sont ensuite transmis via le Bus CAN pour coordonner le déplacement des rétroviseurs. ARD1 assure la synchronisation entre les commandes utilisateur et les mouvements des rétroviseurs, contribuant ainsi à une expérience de conduite plus personnalisée et confortable.

d) Intégration dans le système global :

L'intégration de ARD1 dans le système global est un élément clé pour assurer le bon fonctionnement du projet. En tant que microcontrôleur central, ARD1 agit comme l'interface utilisateur principale, gérant les commandes manuelles via les boutons-poussoirs et affichant des informations sur l'afficheur LCD. Son intégration avec le module MCP2515 lui permet de communiquer efficacement avec d'autres composants du système via le Bus CAN. ARD1 joue un rôle central en traitant les commandes, en transmettant des signaux au MCP2515, et en contribuant ainsi au contrôle synchronisé des rétroviseurs de la voiture. Cette intégration harmonieuse dans le système global garantit une communication fluide entre tous les éléments, offrant une expérience utilisateur cohérente et fiable.

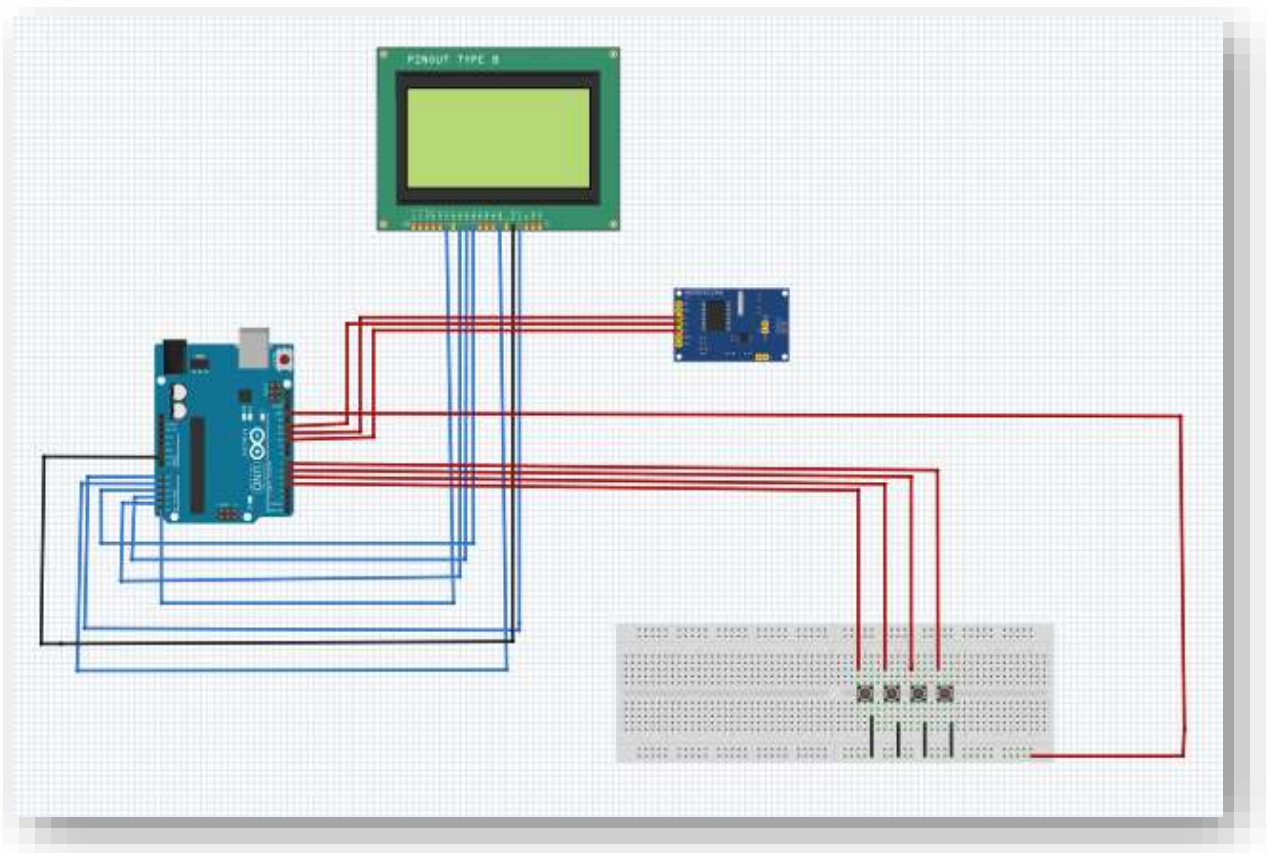


Figure 19: Partie 'write'

2. Description du microcontrôleur ARD2 :

ARD2, représentant Arduino 2 dans le cadre du projet, assume une fonction cruciale dans le contrôle des moteurs des rétroviseurs de la voiture. Choisi pour sa compatibilité avec le Bus CAN et son intégration harmonieuse dans le système, ARD2 sert de lien entre les commandes utilisateur et le mouvement des moteurs.

c) Capacité de communication avec le BUS CAN :

L'intégration du module MCP2515 permet à ARD2 d'établir une communication bidirectionnelle avec le Bus CAN. En tant que transmetteur/récepteur CAN, le MCP2515 facilite l'échange de données entre ARD2 et le Bus CAN, permettant ainsi la réception des commandes de rétroviseurs et l'envoi d'informations sur l'état du système.

b) Traitement des commandes et contrôle des moteurs des rétroviseurs :

Le traitement des commandes et le contrôle des moteurs des rétroviseurs sont au cœur du système. En interprétant les signaux du Bus CAN, ARD2 orchestre le mouvement précis des moteurs pour répondre aux commandes de l'utilisateur. Cette synchronisation entre les commandes et le contrôle des moteurs assure une expérience de conduite personnalisée et réactive.

c) Intégration dans le système global :

ARD2 s'intègre de manière transparente dans l'ensemble du système, agissant en tandem avec ARD1 pour garantir une communication fluide. Grâce à son lien avec le Bus CAN, ARD2 contribue activement à l'efficacité du système global en assurant la cohérence entre les commandes utilisateur, le traitement des signaux et le mouvement des moteurs des rétroviseurs.

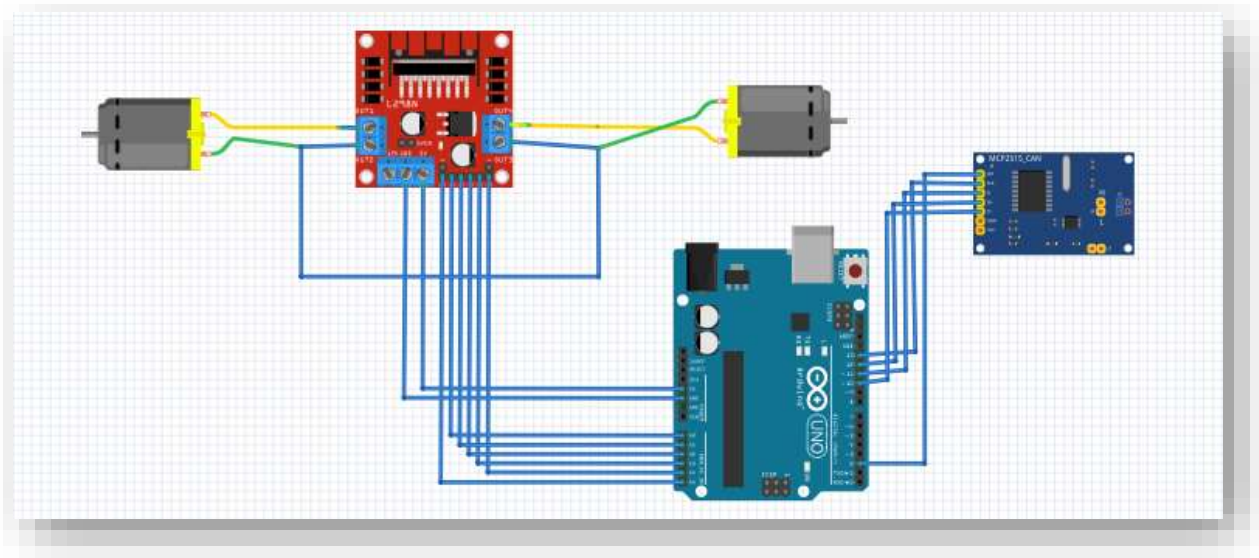


Figure 20: Partie 'read'

3. Schéma de connexion :

Le schéma de connexion illustre la configuration électrique du projet et la manière dont les différents composants interagissent entre eux :

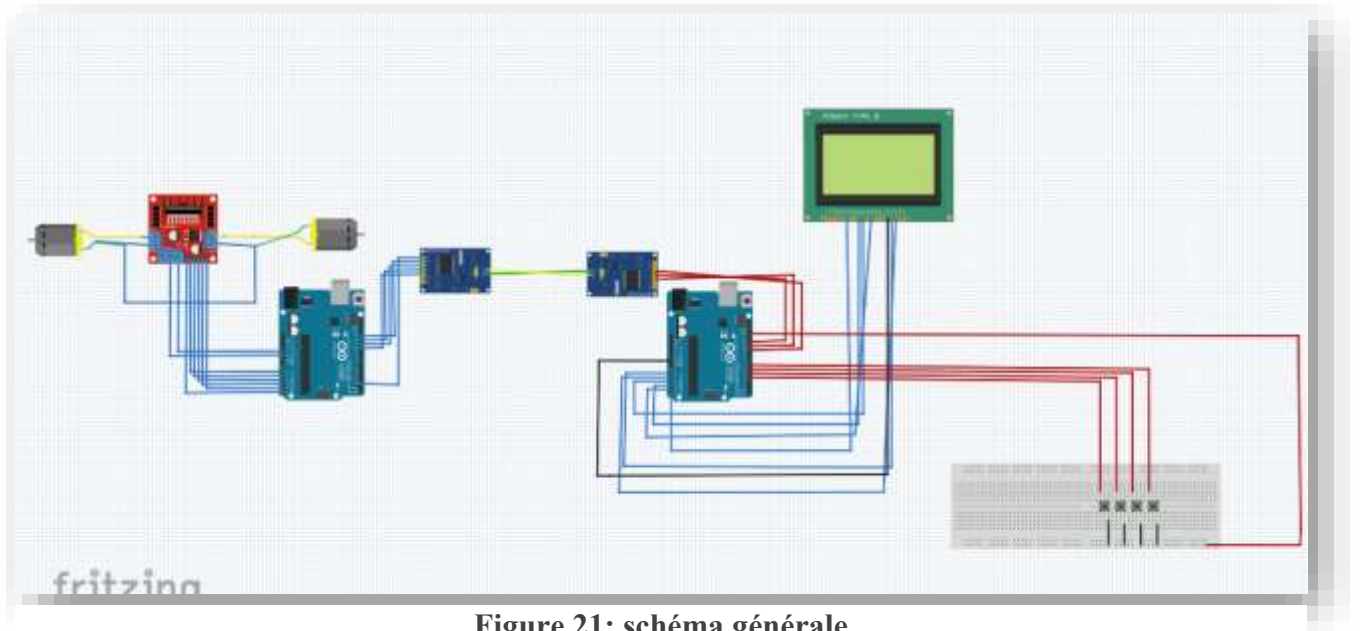


Figure 21: schéma générale

Sur l'annexe 1 :

Partie1 : nous présentons le Schéma de Projet dans Proteus.

Partie2 : nous présentons le projet en pratique.

II. Programme du système :

1. Programme du microcontrôleur (Write) ARD1 :

- Déclaration :

```
CAN_write
#include <LiquidCrystal.h>
#include <SPI.h>
#include <mcp2515.h>

const int rs = A0, en = A1, d4 = A2, d5 = A3, d6 = A4, d7 = A5;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

#define CS_PIN 10
MCP2515 mcp2515(CS_PIN);

#define UP_BUTTON_PIN 7
#define DOWN_BUTTON_PIN 6
#define LEFT_BUTTON_PIN 5
#define RIGHT_BUTTON_PIN 4
#define switch_right 1
#define switch_left 0

struct can_frame canMsg1;
```

- Setup :

```
void setup() {
    Serial.begin(115200);
    while (!Serial);

    canMsg1.can_id = 0x101;
    canMsg1.can_dlc = 1;
    canMsg1.data[0] = 0x01;
    canMsg1.data[1] = 0x02;
    canMsg1.data[2] = 0x03;
    canMsg1.data[3] = 0x04;
    canMsg1.data[4] = 0x05;
    canMsg1.data[5] = 0x06;
    canMsg1.data[6] = 0x07;
    canMsg1.data[7] = 0x08;
    mcp2515.reset();
    mcp2515.setBaudrate(CAN_125KBPS);
    mcp2515.setNormalMode();
    Serial.println("Example: Write to CAN");

    pinMode(UP_BUTTON_PIN, INPUT_PULLUP);
    pinMode(DOWN_BUTTON_PIN, INPUT_PULLUP);
    pinMode(LEFT_BUTTON_PIN, INPUT_PULLUP);
    pinMode(RIGHT_BUTTON_PIN, INPUT_PULLUP);
    pinMode(switch_right, INPUT_PULLUP);
    pinMode(switch_left, INPUT_PULLUP);
    lcd.begin(20, 4);
}
```

- **Loop :**

```
void loop() {
    lcd.setCursor(0,0);
    lcd.print("CONTROLE: ");
    canMsg1.can_id = 0x101;
    lcd.setCursor(0,2);
    lcd.print("BUTTON: ");
    if (digitalRead(switch_left) == 0 || digitalRead(switch_right) == 0) {
        if (digitalRead(switch_left) == 0 ) {
            lcd.setCursor(0,1);
            lcd.print(" Mirror Left");
        }
        else{
            lcd.setCursor(0,1);
            lcd.print(" Mirror Right");
        }
        buttonUp();
        buttonDown();
        buttonLeft();
        buttonRight();
        delay(100);
    }
    else{
        lcd.setCursor(0, 3);
        lcd.print("No message found ");
    }
}
```

- **Fonction Button Up :**

```
void buttonUp() {
    if (digitalRead(UP_BUTTON_PIN) == 0) {
        if (digitalRead(switch_left) == 0) {
            canMsg1.data[0] = 0x01;
        } else {
            canMsg1.data[0] = 0x05;
        }
        mcp2515.sendMessage(&canMsg1);
        Serial.println("Messages UP");
        lcd.setCursor(0, 3);
        lcd.print(" message UP ");
        delay(1000);
        lcd.clear();
    }
}
```

- **Fonction Button Down :**

```
void buttonDown() {  
    if (digitalRead(DOWN_BUTTON_PIN) == 0) {  
        if (digitalRead(switch_left) == 0) {  
            canMsg1.data[0] = 0x02;  
        } else {  
            canMsg1.data[0] = 0x06;  
        }  
        mcp2515.sendMessage(&canMsg1);  
        Serial.println("Messages DOWN");  
        lcd.setCursor(0, 3);  
        lcd.print("    message DOWN    ");  
        delay(1000);  
        lcd.clear();  
    }  
}
```

- **Fonction Button Left :**

```
void buttonLeft() {  
    if (digitalRead(LEFT_BUTTON_PIN) == 0) {  
        if (digitalRead(switch_left) == 0) {  
            canMsg1.data[0] = 0x03;  
        } else {  
            canMsg1.data[0] = 0x07;  
        }  
        mcp2515.sendMessage(&canMsg1);  
        Serial.println("    Messages LEFT    ");  
        lcd.setCursor(0, 3);  
        lcd.print("    message LEFT    ");  
        delay(1000);  
        lcd.clear();  
    }  
}
```

- **Fonction Button Right :**

```
void buttonRight() {  
    if (digitalRead(RIGHT_BUTTON_PIN) == 0) {  
        if (digitalRead(switch_left) == 0) {  
            canMsg1.data[0] = 0x04;  
        } else {  
            canMsg1.data[0] = 0x08;  
        }  
        mcp2515.sendMessage(&canMsg1);  
        Serial.println("Messages RIGHT");  
        lcd.setCursor(0, 3);  
        lcd.print("    message RIGHT    ");  
        delay(1000);  
        lcd.clear();  
    }  
}
```

2. Programme du microcontrôleur (Read) ARD2 :

- **Déclaration :**

```
CAN_read  
#include <SPI.h>  
#include <mcp2515.h>  
  
#define CS_PIN 10 // Chip Select pin for MCP2515  
MCP2515 mcp2515(CS_PIN);  
  
#define MIRROR_UP_COMMAND 0x01  
#define MIRROR_DOWN_COMMAND 0x02  
#define MIRROR_LEFT_COMMAND 0x03  
#define MIRROR_RIGHT_COMMAND 0x04  
  
// Motor A  
#define enA 9  
#define in1 A0  
#define in2 A1  
  
// Motor B  
#define enB 3  
#define in3 A2  
#define in4 A3  
  
struct can_frame canMsg;
```

- Setup :

```
void setup() {
  Serial.begin(115200);

  mcp2515.reset();
  mcp2515.setBtrrate(CAN_125KBPS);
  mcp2515.setNormalMode();

  Serial.println("----- CAN Read -----");
  Serial.println("ID   DLC   DATA");

  // Setup motor pins
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(enB, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
}
```

- Loop :

```
void loop() {
  if (mcp2515.readMessage(&canMsg) == MCP2515::ERROR_OK) {
    Serial.print(canMsg.can_id, HEX); // print ID
    Serial.print(" ");
    Serial.print(canMsg.can_dlc, HEX); // print DLC
    Serial.print(" ");
    for (int i = 0; i < canMsg.can_dlc; i++) { // print the data
      Serial.print(canMsg.data[i], HEX);
      Serial.print(" ");
    }
    Serial.println();
    // Check the CAN ID and process the message
    if (canMsg.can_id == 0x101) {
      // Message from Arduino 2 to control Motor 1
      if (canMsg.data[0] == MIRROR_UP_COMMAND) {
        MIRROR_UP();
        Serial.println("Message UP ");
      } else if (canMsg.data[0] == MIRROR_DOWN_COMMAND) {
        MIRROR_DOWN();
        Serial.println("Message DOWN ");
      } else if (canMsg.data[0] == MIRROR_LEFT_COMMAND) {
        MIRROR_LEFT();
        Serial.println("Message LEFT ");
      } else if (canMsg.data[0] == MIRROR_RIGHT_COMMAND) {
        MIRROR_RIGHT();
        Serial.println("Message RIGHT ");
      }
    }
  }
}
```

- **Fonction MIRROR_UP :**

```
void MIRROR_UP() {  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 200);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 200);  
    delay(500);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 0);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 0);  
}
```

- **Fonction MIRROR_DOWN :**

```
void MIRROR_DOWN() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    analogWrite(enA, 200);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, HIGH);  
    analogWrite(enB, 200);  
    delay(500);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 0);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 0);  
}
```

- **Fonction MIRROR_LEFT :**

```
void MIRROR_LEFT() {  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 200);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, HIGH);  
    analogWrite(enB, 200);  
    delay(500);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 0);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 0);  
}
```

- **Fonction MIRROR_DOWN :**

```
void MIRROR_RIGHT() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    analogWrite(enA, 200);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 200);  
    delay(500);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    analogWrite(enA, 0);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
    analogWrite(enB, 0);  
}
```

CHAPITRE IV

● LA RÉALISATION

ET CIRCUIT IMPRIMÉ

I. La Réalisation du ADR1 :

1. Introduction :

La conception d'un circuit imprimé (PCB) pour un projet Arduino est une étape essentielle pour convertir un prototype en une solution durable et fiable. Dans ce projet particulier, nous nous concentrerons sur la création d'un PCB permettant de contrôler des boutons, un écran LCD, et un switch à l'aide d'un microcontrôleur Arduino.

Le logiciel utilisé dans cette conception est Eagle, un outil puissant de conception de circuits imprimés développé par Autodesk, reconnu pour sa robustesse et sa flexibilité dans la création de schémas électroniques et de layouts de PCB.

2. Schématique de circuit imprimé :

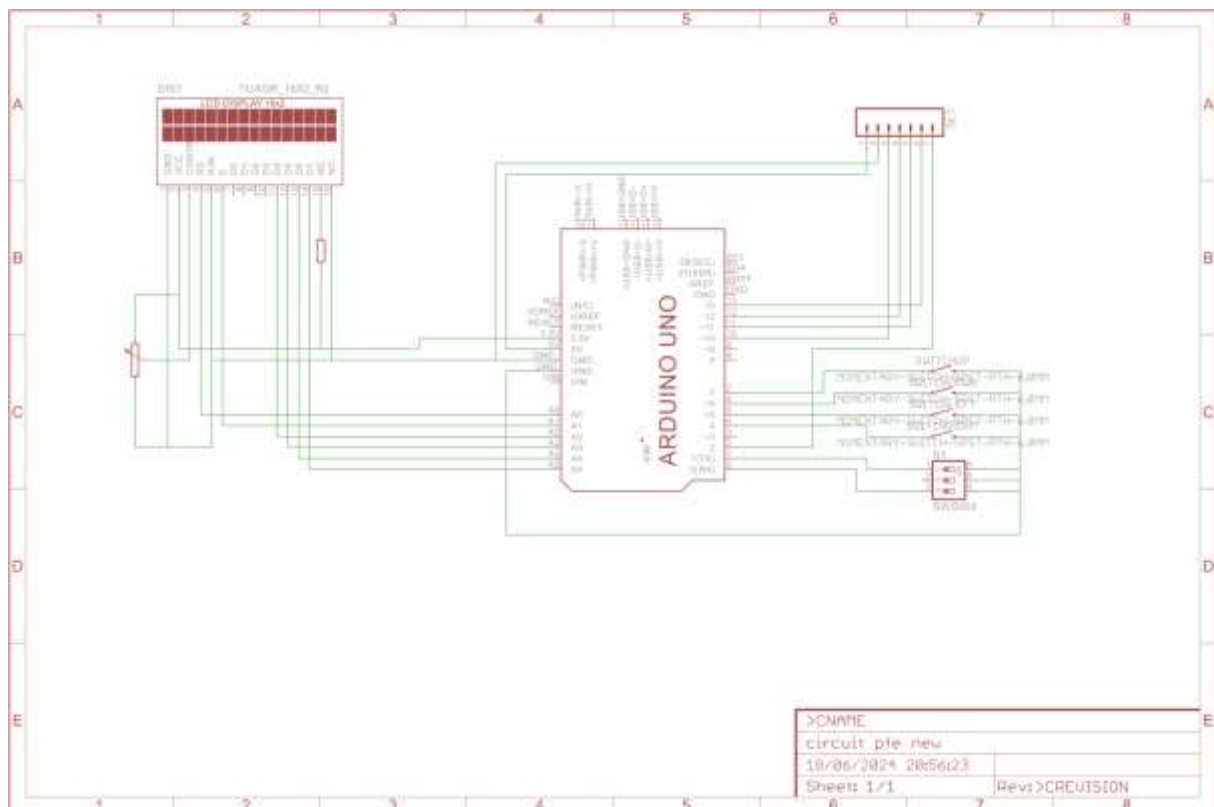


Figure 22: Schématique ARD1

3. Board de circuit imprimé :

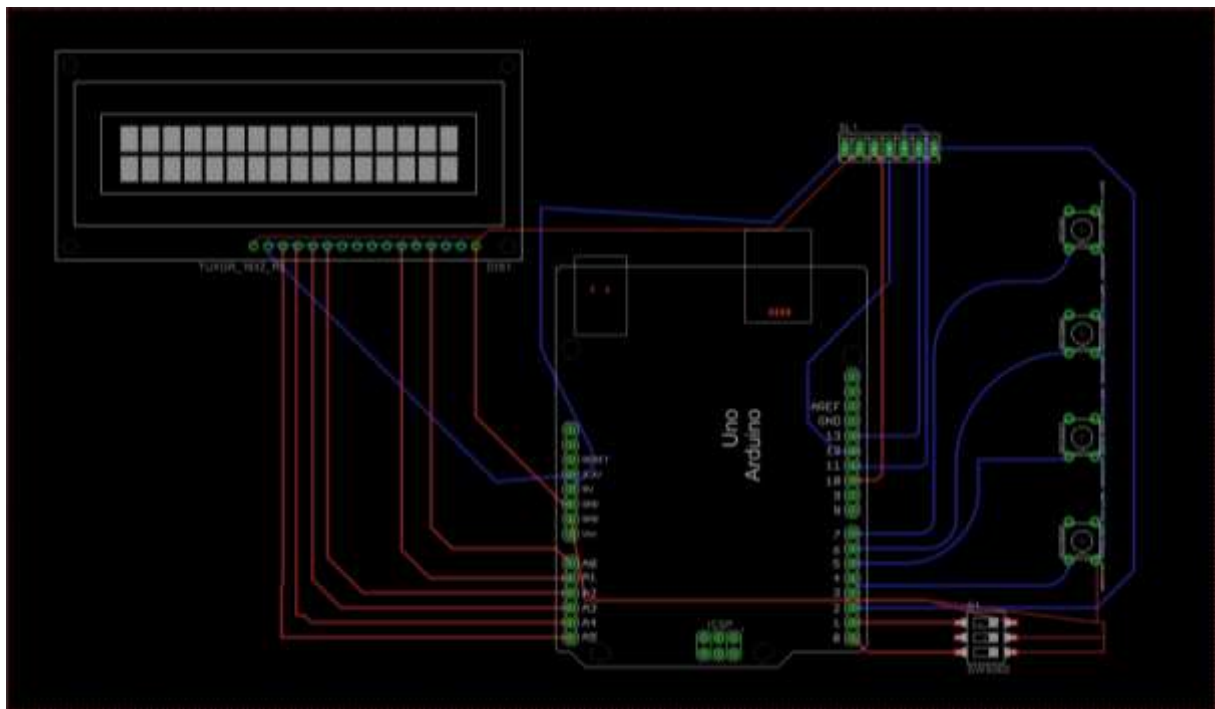


Figure 23: Board ARD1

4. Circuit imprimé :

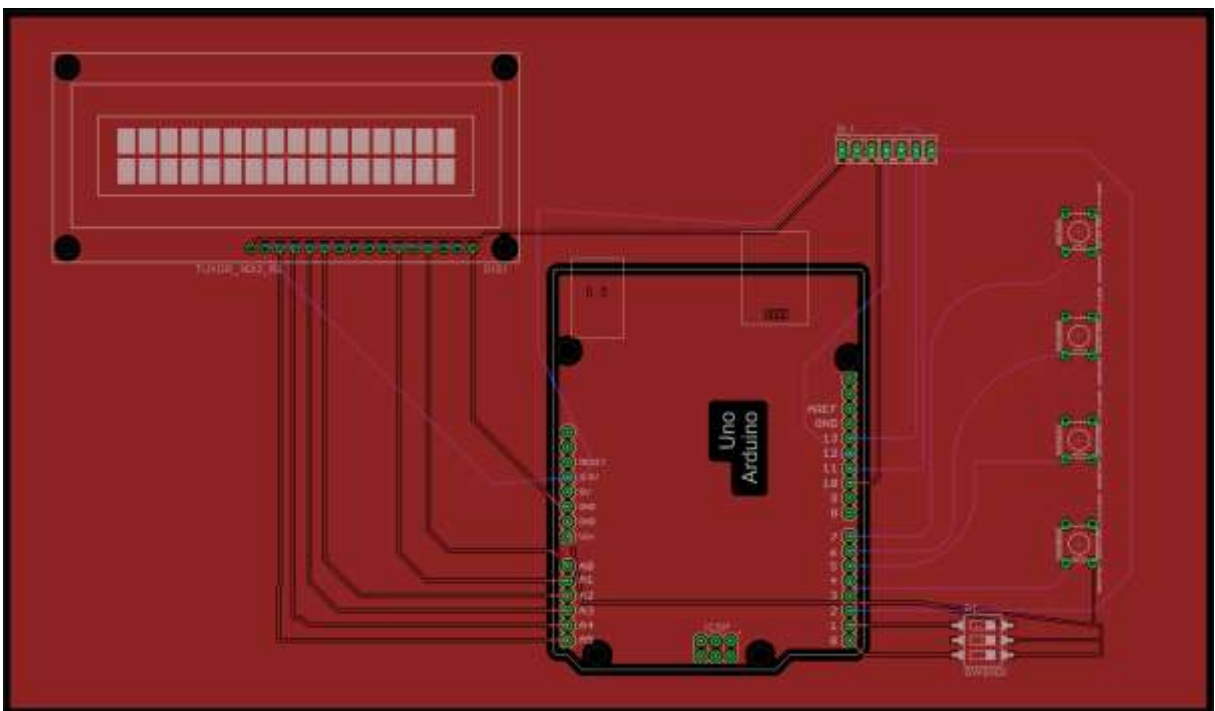


Figure 24: Circuit imprimé de ARD1

II. La Réalisation du ARD2 :

1. Introduction :

Parmi les voitures équipées d'un rétroviseur électronique, notre choix s'est porté sur la voiture Dacia en raison de la disponibilité de cette technologie et de la facilité de manipulation de ses composants électroniques. Le rétroviseur latéral Dacia peut être commandé via de petits moteurs, ce qui permet d'ajuster précisément l'angle de vue nécessaire pour compléter ce projet scolaire axé sur le bus can.

2. Choix de miroirs :

- **Description générale :**

- Composant : Rétroviseur électrique
- Côté : Droit (côté passager)
- Constructeur : Dacia
- Modèle de véhicule : Logan (LS)
- Années de production : 2004 - 2012

- **Spécifications techniques :**

- type de moteur compatible : Essence
- Moteur : 1.6 (LSOB, LSOD, LSOF, LSOH)
- Code moteur : K7M 710
- Puissance moteur : 87 chevaux (HP), 64 kilowatts (KW)
- Cylindrée : 1598 cm³
- Fabricant de la pièce : ZAMAK
- Code OEM (Original Equipment Manufacturer) : 90D13349002, 90D 13349 002



Figure 25: Rétroviseur

- **Forme intérieure :**

- 1) **Emplacement des moteurs :**



Figure 26: Emplacement des moteurs

- 2) **Câblage avec Les Moteur DC :**



Figure 27: Câblage avec Les Moteur DC

CHAPITRE **V** . LES LOGICIELS UTILISÉS

I. Proteus8 Professional 'ISIS' :

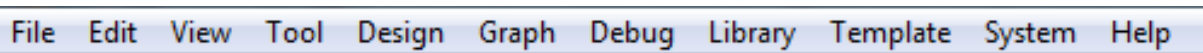
Proteus 8 Professional 'ISIS' est un logiciel de conception de circuits électroniques développé par Labcenter Electronics. ISIS, acronyme pour Intelligent Schematic Input System, est une partie essentielle de ce logiciel, dédiée à la saisie et à la simulation de schémas électroniques. Ce logiciel propose une interface conviviale pour dessiner des schémas, une bibliothèque de composants extensible incluant des microcontrôleurs, des composants analogiques et numériques, et des outils de conception avancés pour faciliter la création de circuits complexes. La simulation en temps réel permet de vérifier le comportement des circuits avant leur fabrication, avec un débogage interactif pour valider et corriger les erreurs. Proteus 8 Professional s'intègre également avec ARES (Advanced Routing and Editing Software) pour la conception de circuits imprimés (PCB), offrant des outils de routage automatique et manuel ainsi qu'une visualisation 3D des PCB. Les outils d'analyse, tels que l'analyseur logique et l'oscilloscope virtuel, aident à tester et surveiller les signaux du circuit. Ce logiciel est largement utilisé dans l'éducation, la recherche et développement, et le prototypage rapide, permettant de réduire le temps de développement et d'augmenter la précision des conceptions.

- En somme, Proteus 8 Professional 'ISIS' est une solution puissante et polyvalente pour les concepteurs de circuits électroniques, transformant efficacement des idées en réalité.



Figure 28: ISIS

1. Barre de menus



Cette barre permet de gérer votre travail (ouverture, sauvegarde, impression, mode d'affichage, etc.).

2. Barres d'outils de commande

Ces barres fournissent un accès équivalent aux commandes des menus. Elles peuvent être masquées par la commande "Barre d'outils" du menu "Affichage"



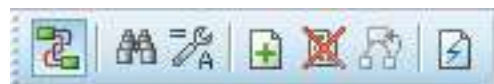
→ Commandes Fichier / Projet



→ Commande d'affichage



→ Commandes Édition / Bibliothèque



→ Commandes Outils

3. Barre d'outils de sélection de mode

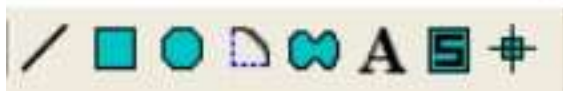
Cette barre permet de sélectionner un outil parmi les 3 modes d'édition disponibles.



→ Mode principal



→ Mode gadgets



→ Mode graphique

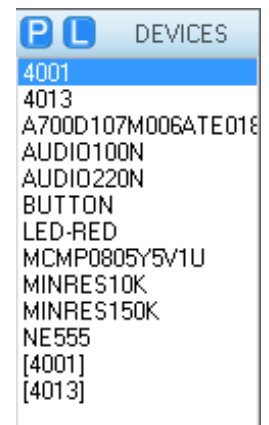
4. Barre d'outils d'orientation



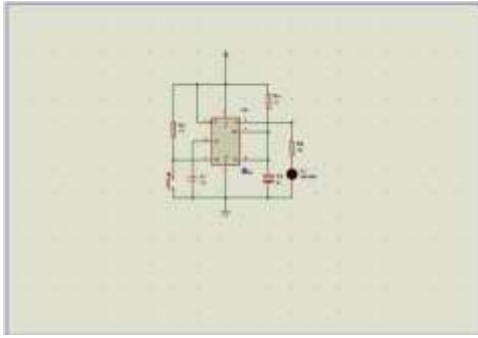
→ Cette barre permet d'afficher et de contrôler la rotation et la réflexion d'un objet placé ou à placer.

5. Sélecteur d'objets

→ Le sélecteur d'objets liste les différents éléments, selon le mode de travail choisi. Les types d'objets qui peuvent y apparaître sont les composants, les terminaux, les pattes, les symboles graphiques, les marqueurs, les graphes.



6. Zone de travail ou d'édition des schémas



→Cet espace rectangulaire correspond à la zone de travail effectif. Tous vos schémas apparaîtront dans cette zone et seront visualisés avec le coefficient d'agrandissement ou de réduction choisi.

II. Arduino IDE :

L'Arduino IDE (Integrated Development Environment) est une plateforme open-source conçue pour la programmation des microcontrôleurs Arduino. Il offre une interface utilisateur simple et intuitive qui facilite la création, l'édition et la gestion de projets électroniques, rendant la programmation accessible aux débutants tout en étant assez puissant pour les utilisateurs avancés. L'IDE prend en charge une large gamme de cartes Arduino et est compatible avec Windows, macOS et Linux. Il comprend des fonctionnalités telles que la compilation rapide, le téléchargement du code via une connexion USB, et un moniteur série intégré pour le débogage en temps réel. Les utilisateurs ont accès à une vaste bibliothèque de composants et de fonctions, ainsi qu'à une documentation complète et une communauté active qui offre des ressources et du soutien. Utilisé dans des domaines allant de l'éducation au prototypage rapide, l'Arduino IDE est un outil essentiel pour transformer des idées en projets électroniques concrets.



Figure 29: Arduino IDE

1. Logiciel :

→En haut du logiciel vous avez un menu avec 5 boutons.

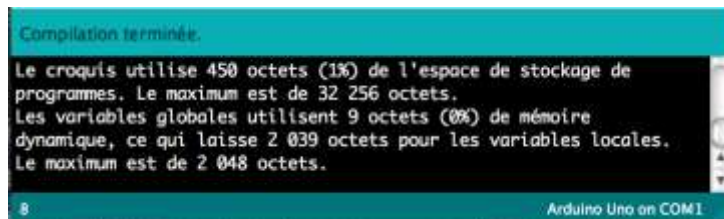


→Ces cinq boutons sont quasiment les seuls que nous aurons à utiliser pour lancer le programme.

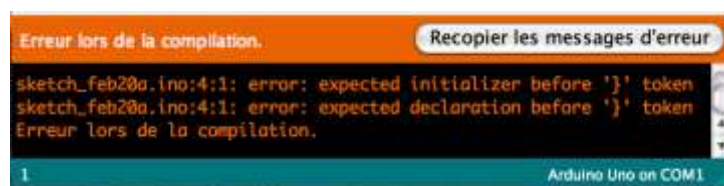


→Ce bouton permet de vérifier votre programme. Le logiciel Arduino va chercher si ce que vous avez écrit est conforme à ce qui est attendu.

→Si tout se passe bien et que votre programme est fonctionnel alors vous devriez avoir l'écran ci-dessous



→Sinon si vous possédez des erreurs dans votre programme la fenêtre suivante apparaît vous indiquant où se situe les erreurs



→Ce bouton permet de **téléverser** le programme. Le logiciel va donc transférer votre programme compilé (transformé en langage machine) dans la mémoire du microcontrôleur de l'Arduino. Une fois cette opération effectuée, l'Arduino gardera ce programme en mémoire et l'exécutera tant qu'il sera alimenté en électricité. Il sera donc autonome et ne dépendra plus de l'ordinateur !

→Les autres boutons représentent :



Créer un nouveau programme



Ouvrir un programme existant



Enregistrer son programme

→Il existe enfin une dernière icône très utile



Elle permet de visualiser soit sous forme graphique soit sous forme de texte les valeurs des capteurs ou de vos variables.

III. Fritzing :

Fritzing est un logiciel open-source de conception de circuits électroniques qui facilite le prototypage et la documentation de projets électroniques. Conçu pour être accessible aux débutants tout en étant utile pour les experts, Fritzing offre une interface intuitive permettant de créer des schémas électroniques, des vues en breadboard, et des conceptions de PCB (circuits imprimés). Il permet aux utilisateurs de passer facilement de la phase de conception sur breadboard à la création de schémas et de PCB prêts pour la fabrication. En plus de son utilité pédagogique dans l'enseignement de l'électronique, Fritzing bénéficie d'une vaste bibliothèque de composants et d'une communauté active qui partage des projets et des idées.

Cette combinaison de fonctionnalités rend Fritzing indispensable pour les hobbyistes, les éducateurs et les ingénieurs cherchant à développer et documenter des projets électroniques de manière efficace et collaborative.

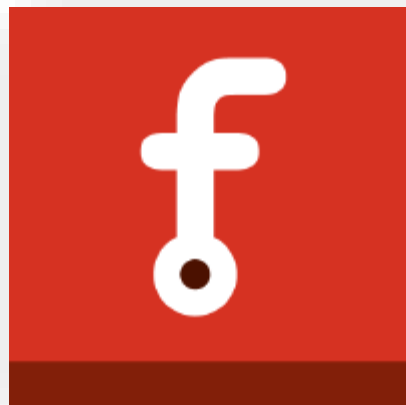


Figure 30: Fritzing

IV. Eagle :

1. L'interface de contrôle d'eagle :

Après le démarrage d'EAGLE, la première fenêtre qui s'ouvre est le panneau de contrôle qui permet :

- ☐ Créer, charger et sauvegarder des projets
- ☐ Configurer certains paramètres de l'application ;
- ☐ Créer, charger et consulter des librairies (Library) de composants.

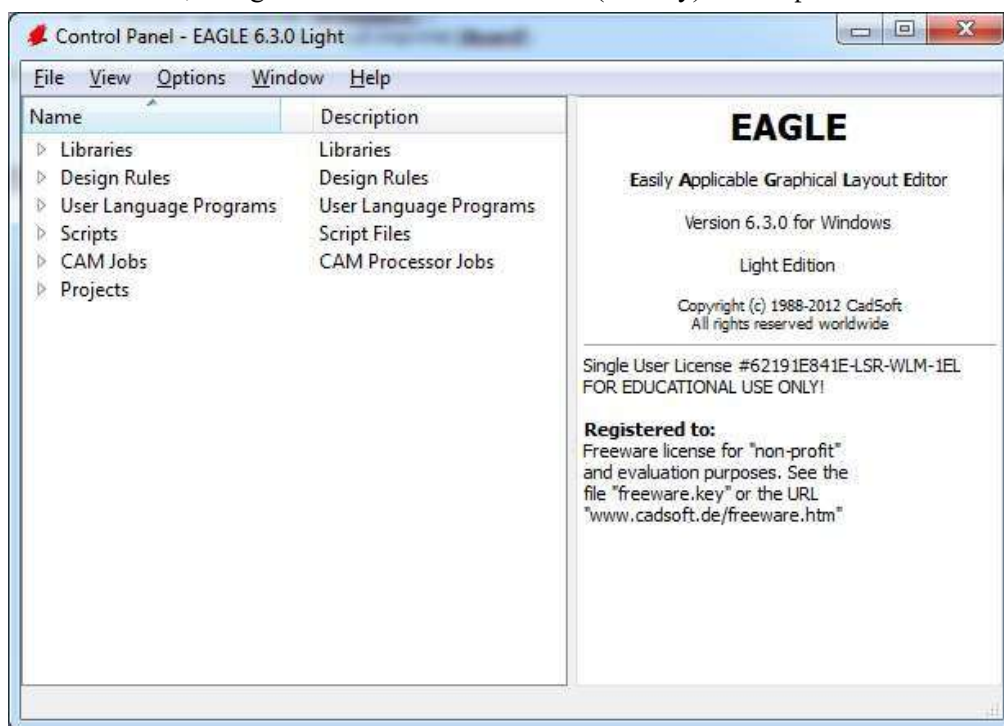


Figure 31: interface du contrôle de eagle

Un projet EAGLE est composé de schémas, de platines et de librairies de composants. La table suivante liste les types de fichiers les plus importants qu'EAGLE peut éditer :

TYPE	FENÊTRE D'EDITION	EXTENSION
Schéma structurel : Schématique	Éditeur de schémas (Schématique Editor)	*.SCH
Platine de circuit imprimé : Board	Éditeur de platines (Lay-out Editor)	*.brd
Librairie de composants : Library	Éditeur de librairies (Library Editor)	*.lbr

2. La fenêtre d'édition de schéma :

Pour créer un nouveau schéma, on utilise la commande **File>New>Schématique** à partir du panneau de contrôle. La fenêtre de l'éditeur de schéma s'ouvre pour nous permettre de commencer la saisie du schéma :

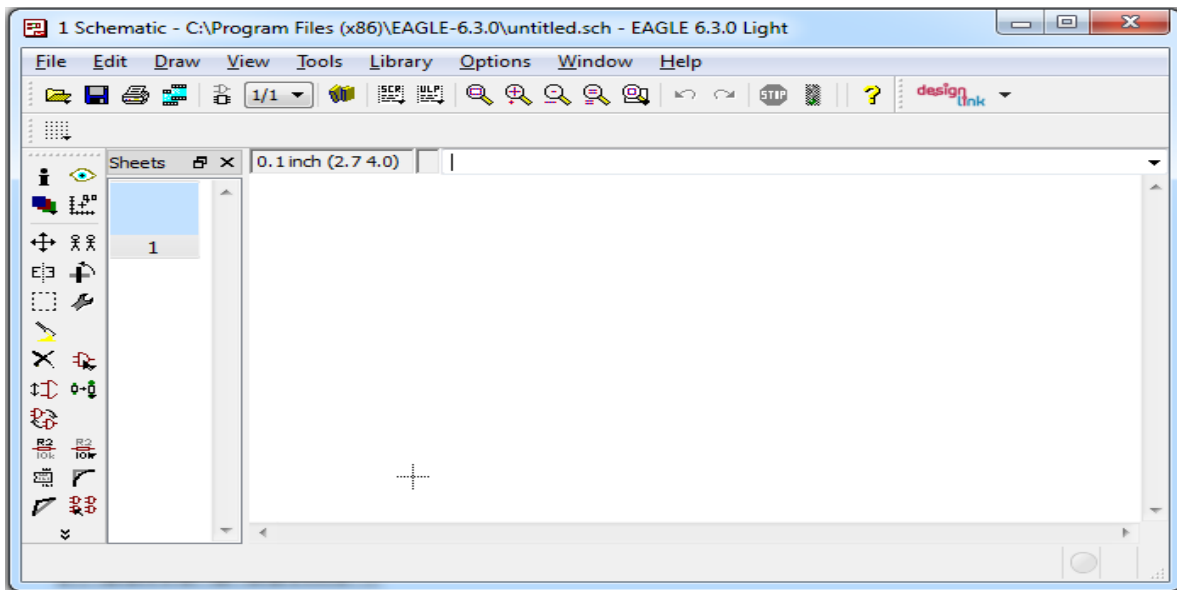


Figure 32: La fenêtre d'édition de schéma

3. Barre d'outils :



4. Saisie du schéma :

Si on veut placer les composants avec précision, on peut activer la grille :

- ✗ Choisir la commande **View>Grid...** ou cliquer sur le bouton

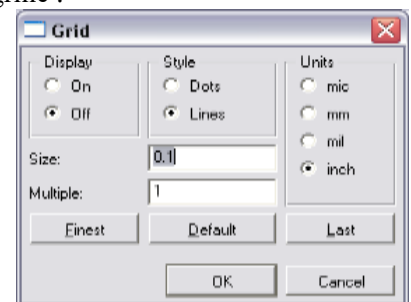


Grid de la barre d'outils.

- ✗ Dans la boîte de dialogue qui apparaît, on peut choisir le pas de la grille **Size**, son unité de mesure **Units**, valider ou non son affichage **Display**, etc.

Pour ajouter les composants du schéma :

- ✗ On clique sur le bouton **Add**  de la barre d'outils à gauche de l'écran.



La boite de dialogue qui apparaît permet de choisir les composants dans les librairies :

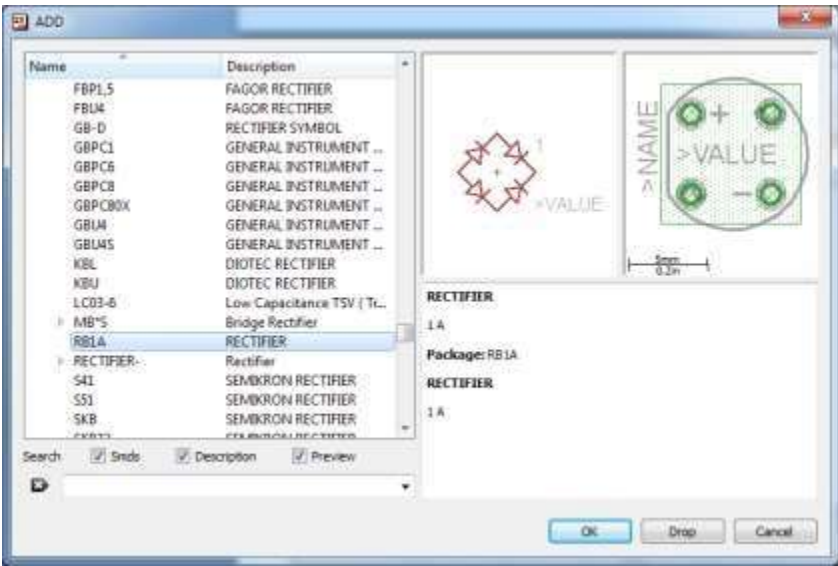


Figure 33: La boite de dialogue

Le tableau suivant résume les autres boutons de la barre d'outils qui peuvent être utilisés :

BOUTON	DESCRIPTION
 : Rotate	Rotation du composant sélectionné
 : Delete	Supprimer le composant sélectionné
 : Name	Renommer le composant sélectionné
 : Value	Changer la valeur du composant sélectionné
BOUTON	DESCRIPTION
 : Invoke	Sélectionner une porte ou les bornes d'alimentation d'un circuit intégré
 : Text	Saisir du texte à placer sur le schéma
 : Junction	Placer un nœud sur le schéma

Le tableau suivant résume les autres boutons de la barre d'outils qui peuvent être utilisés :

BOUTON	DESCRIPTION
 : Auto	Routage automatique du circuit imprimé
 : Ratsnets	Calcule le chemin le plus court pour réaliser une liaison
 : Via	Ajouter une pastille sur le circuit imprimé
 : Rect	Tracer un rectangle avec une couleur donnée pour une éventuelle restriction de tracé

CONCLUSION :

En conclusion, le projet d'interface Bus CAN pour la commande des rétroviseurs d'une voiture a été une opportunité significative pour nous d'approfondir nos connaissances en systèmes électroniques et de mettre en pratique les concepts théoriques appris au cours de notre formation. En exploitant le potentiel du Bus CAN, nous avons conçu un système innovant permettant une gestion centralisée et efficace des rétroviseurs, offrant ainsi aux conducteurs une expérience de conduite plus fluide et sécurisée.

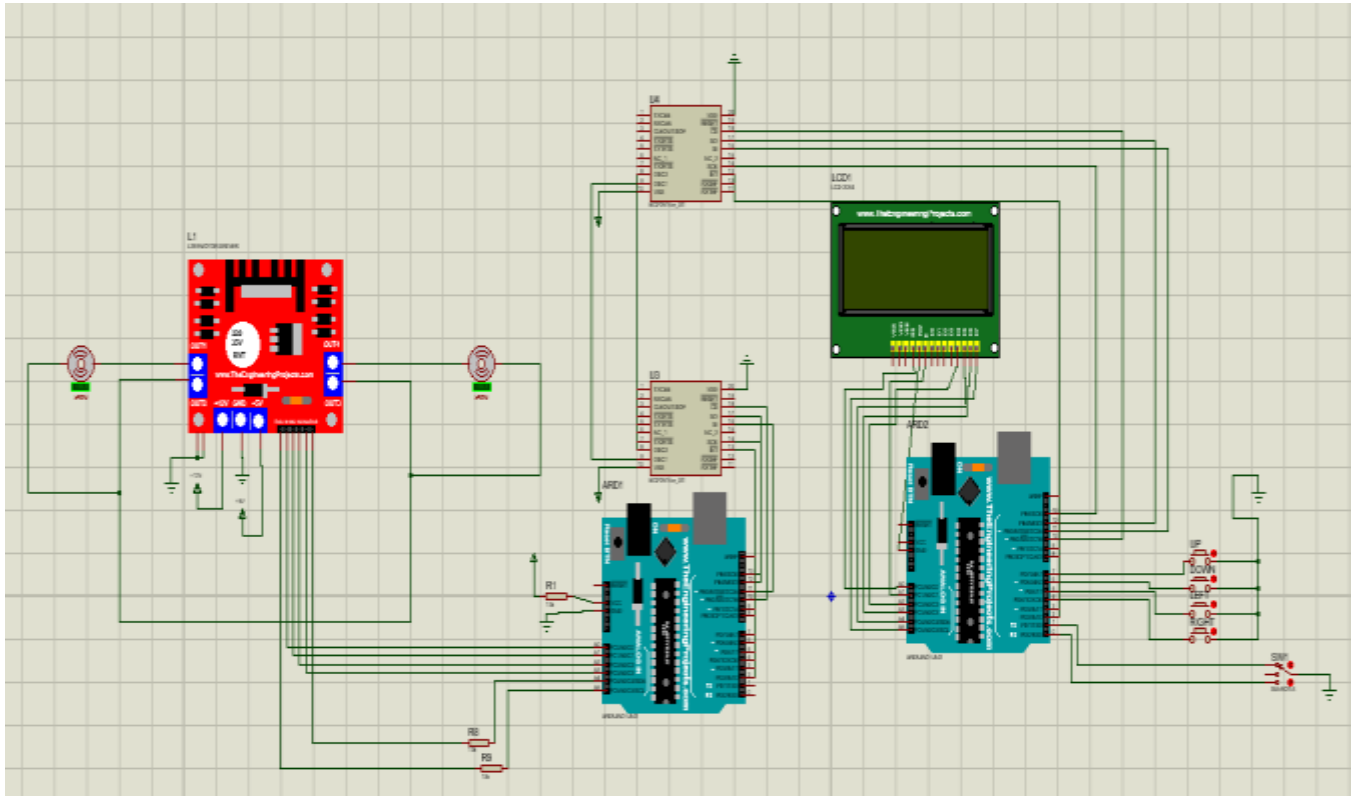
La centralisation de la commande des rétroviseurs via le Bus CAN a permis d'améliorer la réactivité, la précision des ajustements et l'intégration avec d'autres systèmes électroniques du véhicule. Cette approche a démontré les avantages de la communication fiable et de la gestion précise des données offertes par le Bus CAN, soulignant son importance dans l'optimisation des systèmes embarqués automobiles.

En explorant les fonctionnalités du Bus CAN dans le contexte spécifique de la commande des rétroviseurs, nous avons pu identifier les avantages clés de cette technologie, tels que la réduction de la complexité du câblage, l'optimisation de l'efficacité du système et l'intégration harmonieuse avec d'autres composants électroniques du véhicule. Ces éléments ont renforcé notre compréhension des applications pratiques du Bus CAN dans l'industrie automobile et ouvert des perspectives pour de futures innovations dans ce domaine.

En somme, ce projet nous a permis d'acquérir une expérience précieuse en matière de conception, de développement et d'intégration de systèmes électroniques complexes, tout en soulignant l'importance de la communication efficace et de la collaboration entre les différents composants d'un véhicule. Cette expérience nous a non seulement enrichis sur le plan académique, mais elle a également renforcé notre passion pour l'ingénierie et notre engagement envers l'innovation technologique dans le domaine de l'automobile.

annexe 1

Partie1 :



Partie2 :

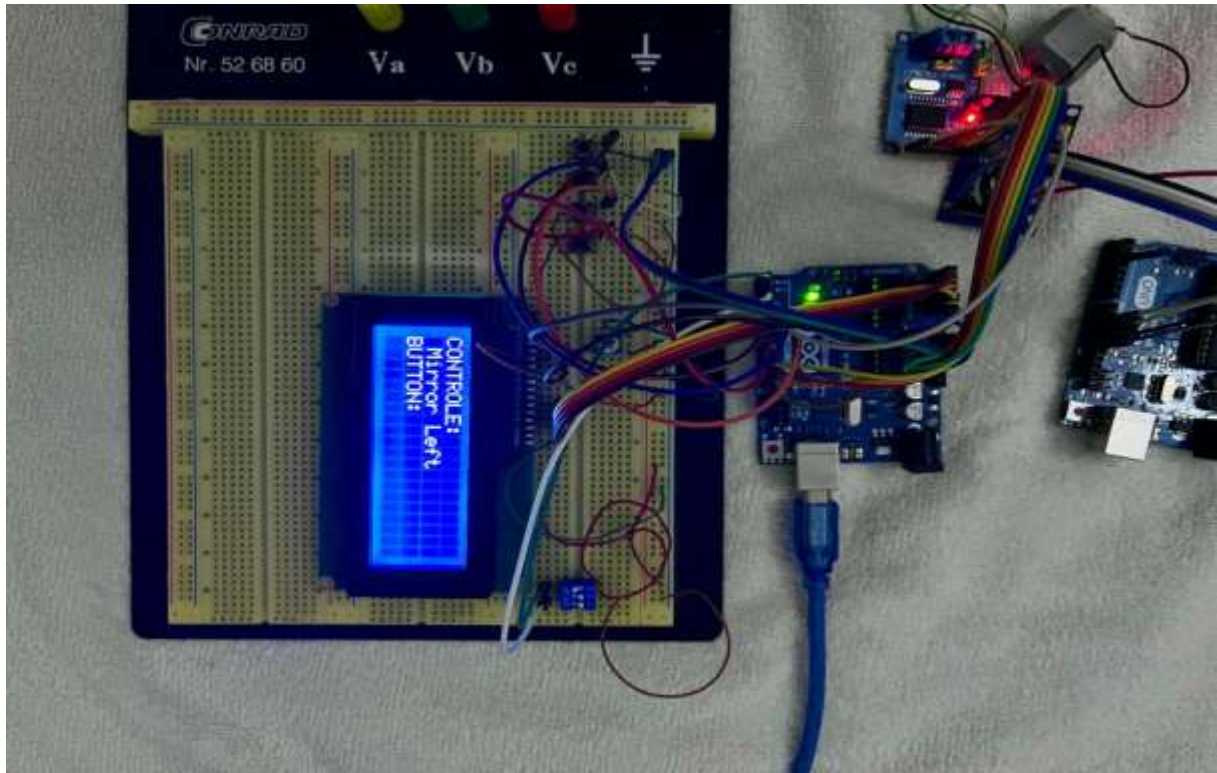


Figure 36: Schéma ARD1

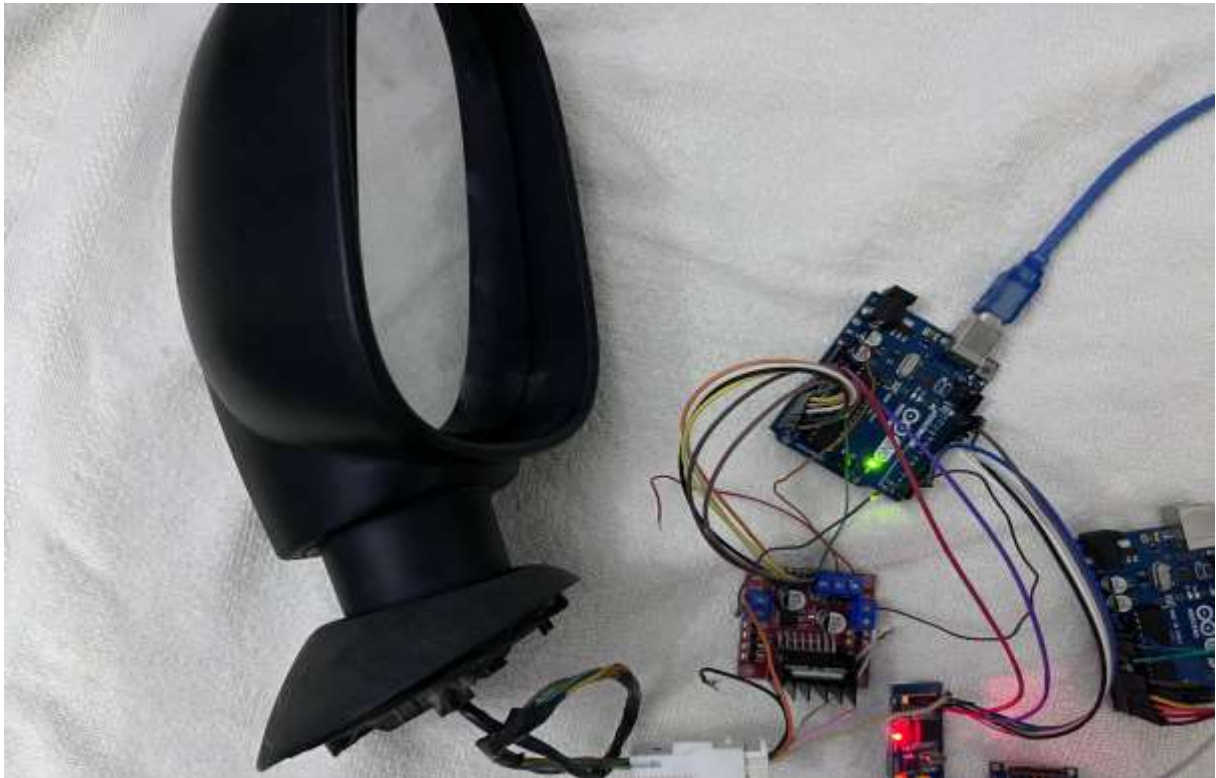


Figure 37: Schéma ARD2



Figure 38: Schéma générale en Pratique

annexe 2

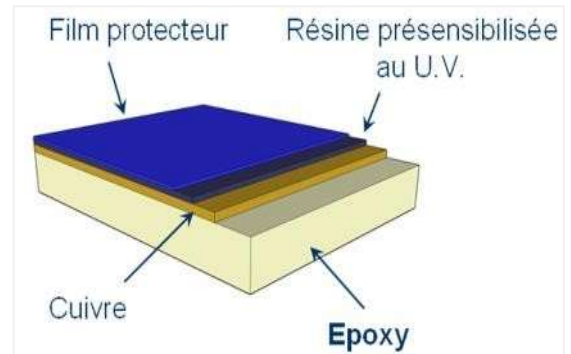
I. Introduction :

Les cartes électroniques se réalisent toujours sur un circuit imprimé, c'est un support isolant sur lequel des pistes de cuivre réalisent la quasi-totalité des connexions entre les composants qu'il supporte. Le circuit imprimé est une platine de matériau composite recouverte d'une (de deux) couche(s) très fine(s) ($35\mu\text{m}$) de cuivre métallique. Le cuivre assure les liens électriques entre les composants, pour que ceux-ci soient conformes au plan du montage.

Le matériau utilisé est presque toujours du verre époxy, c'est un isolant à base de fibres de verre compressées et imprégnées de résine époxy. Une pellicule de cuivre très fine ($< 100\ \mu\text{m}$) est fortement collée sur le support.

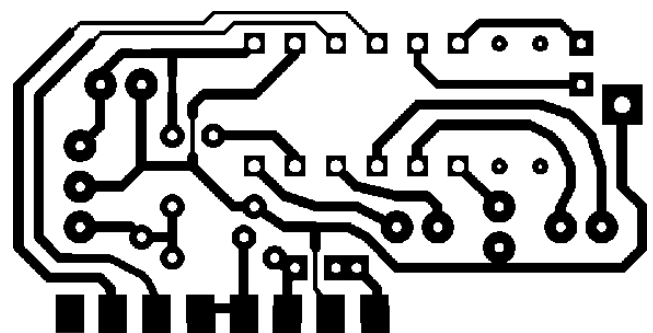
La réalisation des cartes électroniques implique plusieurs étapes, allant de la conception à la fabrication. Voici un aperçu général du processus :

- La réalisation du typon (transparent).
- La découpe.
- L'insolation du circuit imprimé.
- Le développement
- La gravure
- Le perçage
- Les vérifications et tests du circuit imprimé
- La mise en place des composants, leur soudure
- Le test de la carte



II. La réalisation du typon :

Un typon est un masque transparent (papier calque, film plastique) représentant l'implantation structurale d'un circuit électrique tenant compte des dimensions réelles des composants (contrairement au schéma structurel). Il pourra être dessiné à la main sur une feuille de papier calque, ou tracé sur ordinateur à l'aide d'un outil de C.A.O, (logiciel de placement routage), puis imprimé sur un transparent. Dans notre cas on va utiliser le logiciel ARES pour le routage.

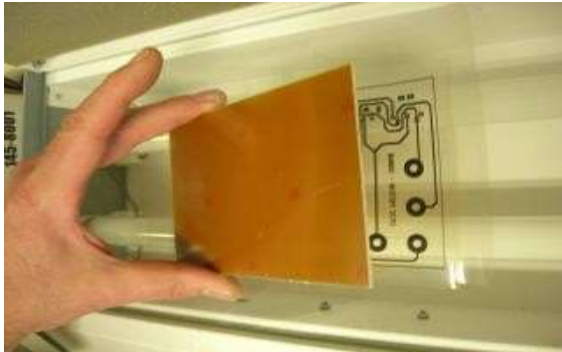


III. La découpe :

Découper l'époxy à la taille du typon. Il est important de limer les bords du circuit, sinon, le bord a de très fortes chances d'être plus haut que le reste de la carte, ce qui rend une insolation nette impossible (les ombres des typons sont floues, les pistes en résultant inutilisables).

IV. L'insolation :

On va placer le typon entre les tubes ultraviolets de l'insoleuse et la résine photosensible. La partie de la résine qui aura été exposée aux ultraviolets va être dégradée et se dissoudra lors de la révélation. L'autre partie, protégée par le typon, restera insensible à la soude.



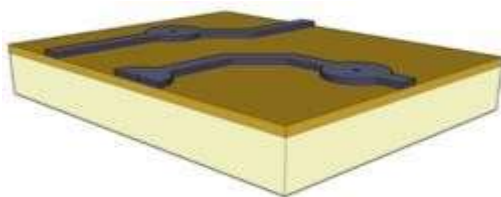
V. La révélation :

On va enlever la résine exposée aux Ultraviolets. Cela ne prend que quelques secondes. Plonger la platine dans un bain de révélateur (de la soude, attention c'est corrosif : manipulez les platines avec des pinces ou avec des gants, ne frottez surtout pas la plaque avec vos doigts, vous effaceriez tout). Agiter le bac 3 ou 4 secondes, le dessin apparaît nettement sur un fond cuivré. Ne surtout pas le laisser trop longtemps, sinon tout partira !

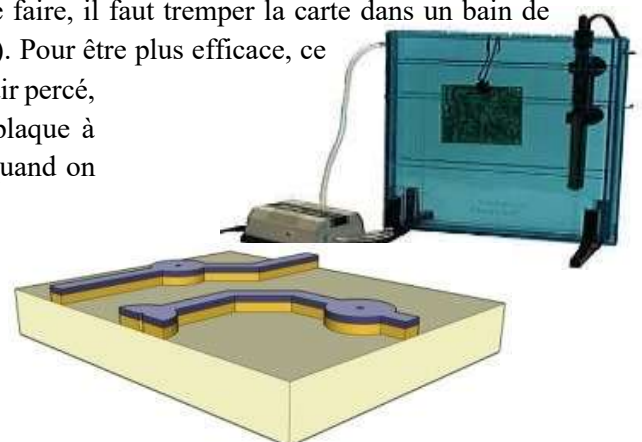


VI. La gravure :

L'étape suivante consiste à ronger le cuivre apparent. A la fin, il ne restera plus que le cuivre protégé. Pour ce faire, il faut tremper la carte dans un bain de gravure (souvent une solution de perchlorure de fer). Pour être plus efficace, ce bain peut être brassé en mettant un tuyau d'arrivée d'air percé, et/ou chauffé. Encore une fois, il faut enlever la plaque à temps (environ 30 min mais disons que c'est bon quand on n'observe plus de cuivre apparent).



Avant la gravure



Après la gravure

VII. Le perçage :

Une fois que la gravure et le nettoyage sont terminés, vous obtenez sur la plaque des pistes de cuivre représentant le tracé du typon. Il faut maintenant percer la plaque aux points où devront être introduits et soudés les composants. Le diamètre de perçage dépend de la taille des pattes des composants. Il existe 3 dimensions de pattes de composants :

- Pour les petits composants (circuits imprimés, fils, etc.) on percera avec un foret de 0,6 mm à 0,8 mm
- Pour les composants moyens (résistances, condensateurs, certaines diodes, LED, etc.) on percera avec un foret de 0,8 mm à 1 mm
- Pour les gros composants (relais, bornier de raccordement, etc.) on percera avec un foret de 1 mm à 1,2 mm

Avant de commencer à souder les composants sur le circuit imprimé, il faut tester la continuité des différentes pistes, à la recherche d'éventuelles microcoupures (des coupures microscopiques dans la piste de cuivre qui ne se voit pas à l'œil nu), qui empêchent tout de même le passage du courant. Pour tester la continuité des pistes, on utilisera un ohmmètre.



VIII. Le soudage :

Généralement On commence par les composants peu encombrants, pour ne pas être gêné pour les soudures suivantes. L'opération de soudure se fait par placer la panne du fer à souder de façon à ce qu'elle soit en contact avec le cuivre de la platine et avec la patte du composant. Laisser chauffer ainsi quelques secondes. Ajouter de la soudure au niveau du contact. Laisser la panne encore quelques secondes afin que la soudure s'étale bien. Retirer le fer à souder. La quantité de soudure apportée est un facteur décisif pour la qualité de la soudure. On voit à la forme de la soudure si la quantité est la bonne.

